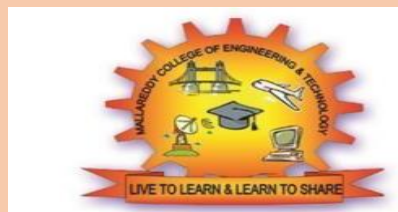


FUNDAMENTALS OF BIG DATA ANALYTICS

(R20A0567)

DIGITAL NOTES

B. TECH IV YEAR – I SEM (2024-2025)



MALLA REDDY COLLEGE OF ENGINEERING & TECHNOLOGY (Autonomous
Institution – UGC, Govt. of India)

Prepared By:

Dr. G. Madhu Mohan

Department of Electrical and Electronics Engineering.

MRCET.

Syllabus :

UNIT I :

INTRODUCTION TO BIG DATA AND ANALYTICS:

Classification of Digital Data, Structured and Unstructured Data - Introduction to Big Data: Characteristics – Evolution – Definition - Challenges with Big Data - Other Characteristics of Data - Why Big Data - Traditional Business Intelligence versus Big Data - Data Warehouse and Hadoop Environment Big Data Analytics: Classification of Analytics – Challenges - Big Data Analytics important - Top Analytics Tools.

UNIT II

INTRODUCTION TO TECHNOLOGY LANDSCAPE: NoSQL, Comparison of SQL and NoSQL, Hadoop -RDBMS Versus Hadoop - Distributed Computing Challenges – Hadoop Overview - Hadoop Distributed File System - Processing Data with Hadoop - Managing Resources and Applications with Hadoop YARN - Interacting with Hadoop Ecosystem

UNIT III

INTRODUCTION TO MONGODB AND MAPREDUCE PROGRAMMING:

MongoDB: Why Mongo DB - Terms used in RDBMS and Mongo DB - Data Types - MongoDB Query Language. MapReduce: Mapper – Reducer – Combiner – Partitioner – Searching – Sorting – Compression.

UNIT IV

INTRODUCTION TO HIVE: Hive: Introduction – Architecture - Data Types - File Formats - Hive Query Language Statements – Partitions – Bucketing – Views - Sub- Query – Joins – Aggregations - Group by and Having - RCFile Implementation - Hive User Defined Function - Serialization and Deserialization.

UNIT V

INTRODUCTION TO DATA ANALYTICS: Machine Learning: Introduction, Supervised Learning, Unsupervised Learning, Machine Learning Algorithms: Regression Model, Clustering, Collaborative Filtering, Associate Rule Making, Decision Tree.

UNIT I

INTRODUCTION TO BIG DATA AND ANALYTICS

- 1. Classification of Digital Data, Structured and Unstructured Data**
- 2. Introduction to Big Data: Characteristics – Evolution – Definition**
- 3. Challenges with Big Data - Other Characteristics of Data**
- 4. Why Big Data**
- 5. Traditional Business Intelligence versus Big Data**
- 6. Data Warehouse and Hadoop Environment Big Data**
- 7. Analytics: Classification of Analytics– Challenges**
- 8. Big Data Analytics important**
- 9. Top Analytics Tools**

INTRODUCTION:

Big data analytics is a field of study and practice that focuses on extracting valuable insights and meaningful patterns from large and complex datasets. It involves the use of various techniques, tools, and technologies to process, analyze, and interpret massive volumes of data to make data-driven decisions, identify trends, and gaining valuable insights.

The term "big data" refers to the vast amounts of structured, semi-structured, and unstructured data that organizations and businesses collect from various sources such as social media, sensors, mobile devices, transaction records, and more. This data is typically characterized by its volume, velocity, variety, and veracity, which makes it challenging to manage and analyze using traditional data processing methods.

Big data analytics encompasses several key components and techniques:

1. **Data collection:** Gathering and aggregating data from multiple sources, including structured databases, log files, social media platforms, and IoT devices.
2. **Data storage and management:** Storing and organizing large volumes of data using distributed file systems, NoSQL databases, and data warehouses.
3. **Data preprocessing:** Cleaning, transforming, and filtering the data to ensure its quality, consistency, and relevance for analysis.
4. **Data analysis:** Applying various statistical, machine learning, and data mining techniques to extract patterns, correlations, and insights from the data. This can involve tasks such as data exploration, clustering, classification, regression, and predictive modelling.
5. **Data visualization:** Presenting the analyzed data in a visual format, such as charts, graphs, and dashboards, to facilitate better understanding and decision-making.
6. **Real-time analytics:** Performing analysis on streaming data to gain immediate insights and enable real-time decision-making.
7. **Data security and privacy:** Ensure appropriate measures are in place to protect sensitive data and comply with relevant regulations and privacy policies.

Data:

Facts that can be recorded, which have implicit meaning.

Ex: Text, Videos, Speech, ELE201, RAM,19 etc.

Big Data:

A massive amount of data which cannot be stored, processed and analyzed using traditional tools is known as Big Data. Hadoop is a framework that stores and processes big data.

Sources of Big Data:

These data come from many sources like

- **Social networking sites:** Facebook, Google, and LinkedIn generate huge amounts of data on a day-to-day basis as they have billions of users worldwide.
- **E-commerce site:** Sites like Amazon, Flipkart, and Alibaba generate many logs from which users buying trends can be traced.
- **Weather Station:** All the weather station and satellite gives huge amounts of data that are stored and manipulated to forecast weather.
- **Telecom company:** Telecom giants like Airtel, and Vodafone study user trends and accordingly publish their plans; for this, they store the data of its million users.
- **Share Market:** Stock exchange worldwide generates huge amounts of data through daily transactions.

Why is Big Data Important?

The importance of big data does not revolve around how much data a company has but how a company utilizes the collected data. Every company uses data in its way; the more efficiently it uses it, the more potential it has to grow. The company can take data from any source and analyze it to find answers which will enable:

- 1. Cost Savings:** Some tools of Big Data like Hadoop and Cloud-Based Analytics can bring cost advantages to businesses when large amounts of data are to be stored, and the tools also help identify more efficient ways of doing business.
- 2. Time Reductions:** The high speed of tools like Hadoop and in-memory analytics can easily identify new data sources, which helps businesses analyse data immediately and make quick decisions based on the learning.
- 3. Understand the market conditions:** By analyzing big data, you can better understand current market conditions. For example, by analyzing customers' purchasing behaviors, a company can find the products that are sold the most and produce products according to this trend. By this, it can get ahead of its competitors.

4. Control online reputation: Big data tools can do sentiment analysis. Therefore, you can get feedback about who is saying what about your company. If you want to monitor and improve the online presence of your business, then big data tools can help in all this

5. Using Big Data Analytics to Boost Customer Acquisition and Retention

The customer is the most important asset any business depends on. No single business can claim success without first having to establish a solid customer base. However, even with a customer base, a business cannot afford to disregard the high competition it faces. If a business is slow to learn what customers seek, it is easy to begin offering poor-quality products. In the end, a loss of clientele will adversely affect business success. Using big data allows businesses to observe various customer-related patterns and trends. Observing customer behaviour is important to trigger loyalty.

6. Using Big Data Analytics to Solve Advertisers Problem and Offer Marketing Insights

Big data analytics can help change all business operations. This includes the ability to match customer expectations, change the company's product line and, of course ensure that the marketing campaigns are powerful

Biga data analytics:

Big data analytics is the process of extracting meaningful insights from big data, such as hidden patterns, unknown correlation, market trends and customer preferences, that can help organizations make informed business decisions.

There are quite a few advantages to incorporating big data analytics into a business or organization. These include:

- **Cost reduction:** big data can reduce costs in storing all the business data in one place. Tracking analytics also helps companies find ways to work more efficiently to cut costs wherever possible.
- **Product development:** Developing and marketing new products, services, or brands is much easier when based on data collected from customers' needs and wants. Big data analytics also helps businesses understand product viability and keep up with trends.
- **Strategic business decisions:** The ability to constantly analyze data helps businesses make better and faster decisions, such as cost and supply chain optimization.

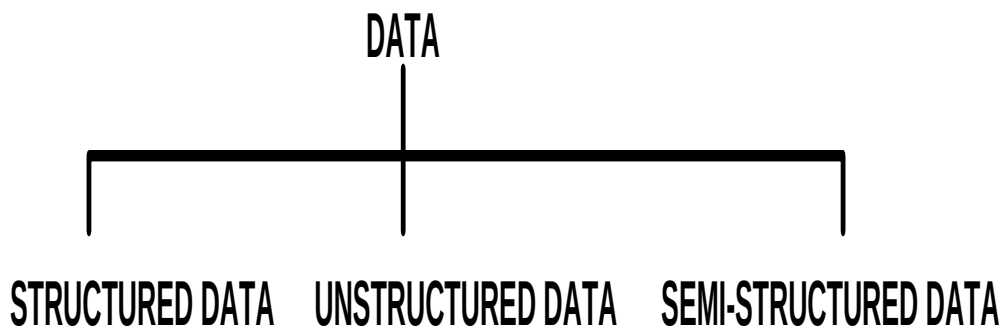
- **Customer experience:** Data-driven algorithms help marketing efforts (targeted ads, for example) increase customer satisfaction by delivering an enhanced customer experience.
- **Risk management:** Businesses can identify risks by analyzing data patterns and developing solutions for managing those risks.

I.Data Classification:

Process of classifying data in relevant categories to be used or applied more efficiently. The classification of data makes it easy for the user to retrieve it. Data classification holds its importance regarding data security and compliance and meeting different business or personal objectives. It is also a major requirement, as data must be easily retrievable within a specific period.

Types of Data Classification:

Data can be broadly classified into 3 types.



1. Structured Data:

- Structured data is created using a fixed schema and is maintained in tabular format. The elements in structured data are addressable for effective analysis.
- It contains all the data which can be stored in the SQL database in a tabular format.
- Today, most data is developed and processed in the simplest way to manage information.

Relational data, Geo-location, credit card numbers, addresses, etc.

- Consider an example of Relational Data you have to maintain a record of students for a university, like a student's name, ID, address, and Email of the student. To store the record of students, the following relational schema and table were used.

S_ID	S_Name	S_Address	S_Email
1001	A	Delhi	A@gmail.com
1002	B	Mumbai	B@gmail.com

2. Unstructured Data:

- It is defined as the data that does not follow a pre-defined standard, or you can say that any does not follow any organized format.
- This kind of data is also not fit for the relational database because you will see a pre-defined manner or organized way of data in the relational database.
- Unstructured data is also very important for the big data domain and to manage and store Unstructured data there are many platforms to handle it like No-SQL Database.

Examples – Word, PDF, text, media logs, etc.

3. Semi-Structured Data:

Semi-structured data is information that does not reside in a relational database but that have some organizational properties that make it easier to analyze. With some process, you can store them in a [relational database](#) but is very hard for some semi-structured data, but semi-structured exist to ease space.

Example – [XML data](#).

Features of Data Classification:

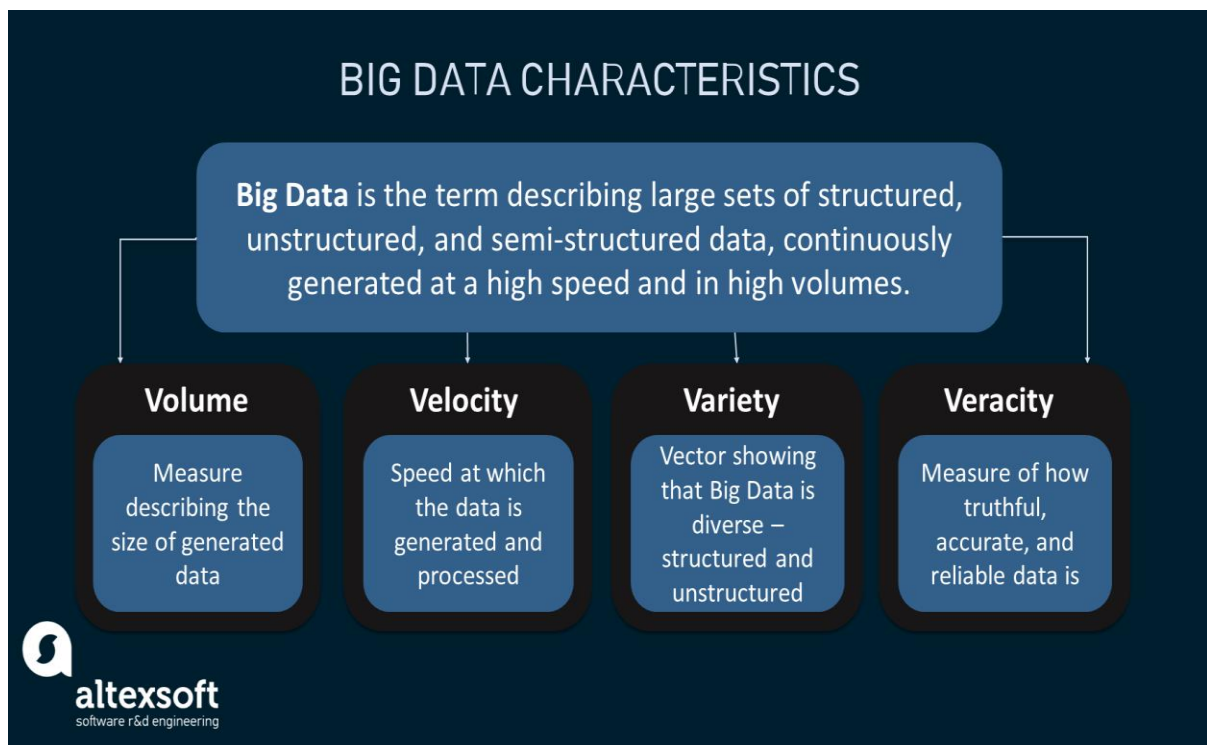
The main goal of the organization of data is to arrange the data in such a form that it becomes fairly available to the users. So, it's basic features as following.

- **Homogeneity** – The data items in a particular group should be similar to each other.
- **Clarity** – There must be no confusion in positioning any data item in a particular group.
- **Stability** – The data item set must be stable i.e. any investigation should not affect the same set of classification.
- **Elastic** – One should be able to change the basis of classification as the purpose of classification changes.

II. Characteristics of Big Data

Big Data contains much data not being processed by traditional data storage or the processing unit. Many multinational companies use it to **process** the data and business of many **organizations**. The data flow would exceed **150 exabytes** per day before replication.

There are five v's of Big Data that explains the characteristics.



5 V's of Big Data

- Volume
- Veracity
- Variety
- Value
- Velocity

1. Volume

- The name Big Data itself is related to its enormous size. Big Data is a vast 'volume' of data generated from many sources daily, such as business processes, machines, social media platforms, networks, human interactions, and many more.

- Facebook can generate approximately a billion messages, 4.5 billion times that the "Like" button is recorded, and more than 350 million new posts are uploaded daily. Big data technologies can handle large amounts of data.

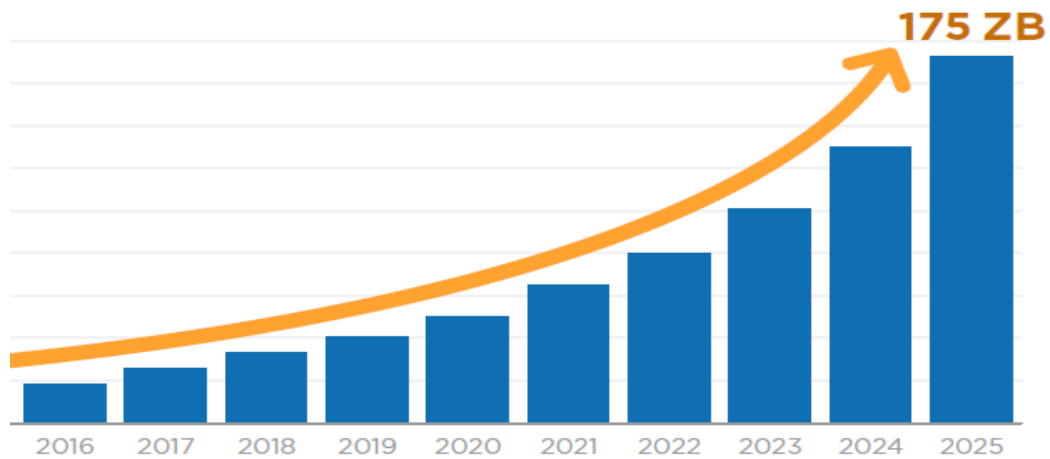


Fig: Expected volume of Big data

2. Variety

Big Data can be structured, unstructured, semi-structured and collected from different sources. Data will only be collected from databases and sheets in the past, But these days the data will come in array forms, PDFs, Emails, audio, SM posts, photos, videos, etc.

3. Veracity

Veracity means how much the data is reliable. It has many ways to filter or translate the data. Veracity is the process of being able to handle and manage data efficiently. Big Data is also essential in business development.

For example, Facebook posts with hashtags.

4. Value

Value is an essential characteristic of big data. It is not the data that we process or store. It is valuable and reliable data that we store, process, and also analyze.

5. Velocity

Velocity plays an important role compared to others. Velocity creates the speed by which the data is created in real-time. It contains the linking of incoming data set speeds, rate of change, and activity bursts. The primary aspect of Big Data is to provide demanding data rapidly.

Big data velocity deals with the speed of data flows from sources like application logs, business processes, networks, social media sites, sensors, mobile devices, etc.

➤ Every one minute

100000 + tweets ,

69500 + status updates ,

11000000 +internet messages ,

698445 +google searches ,

168000000 + emails ,

1820 TB data created

III. History of big data / Evolution of big data

The history of big data can be traced back to the early days of computing and the emergence of digital data. Here's a brief overview of the key milestones in the evolution of big data:

1. **Emergence of Databases:** In the 1960s, the concept of databases and data management systems began to take shape. Developing relational database management systems (RDBMS) like IBM's System R and Oracle's RDBMS laid the foundation for structured data storage and retrieval.
2. **Data Warehousing:** In the 1980s, the concept of data warehousing emerged. Data warehousing involves consolidating data from multiple sources into a central repository for analysis and reporting purposes. This approach enabled organizations to leverage their data assets for decision-making.

3. **Internet and Web 1.0:** The rise and the advent of the World Wide Web in the 1990s brought about a massive increase in data generation. The web became a platform for publishing, sharing, and collecting data, leading to the growth of unstructured and semi-structured data sources.
4. **Digital Transformation and Enterprise Systems:** Organizations started generating vast amounts of structured data with the widespread adoption of enterprise systems such as Enterprise Resource Planning (ERP) and Customer Relationship Management (CRM). This marked the beginning of data-driven decision-making within businesses.
5. **Hadoop and Distributed Computing:** In 2005, Doug Cutting and Mike Cafarella developed Hadoop, an open-source framework designed to process large datasets in a distributed computing environment. Hadoop's distributed file system (HDFS) and MapReduce programming model made it easier to handle big data by breaking it into smaller parts and processing them in parallel.
6. **The Rise of NoSQL:** Traditional relational databases struggled to handle big data's volume, variety, and velocity. Consequently, the development of NoSQL (Not Only SQL) databases gained prominence in the late 2000s. NoSQL databases offered flexible schema designs and horizontal scalability, making them suitable for handling unstructured and semi-structured data.
7. **Advanced Analytics and Machine Learning:** As big data grew, organizations sought to extract valuable insights and patterns from the data. Advanced analytics techniques, such as data mining, machine learning, and predictive modeling, gained traction for uncovering actionable insights and supporting decision-making.
8. **Cloud Computing:** The advent of cloud computing in the late 2000s provided organizations with scalable and cost-effective infrastructure for storing, processing, and analyzing big data. Cloud platforms, such as Amazon Web Services (AWS) and Google Cloud Platform (GCP), offered on-demand resources, eliminating the need for large-scale on-premises infrastructure.
9. **Internet of Things (IoT):** With the proliferation of connected devices and sensors, the amount of data generated by IoT devices skyrocketed. The IoT contributed to the expansion of big data, as real-time streaming data from various sources became available for analysis.
10. **Current Trends:** Today, big data analytics continues to evolve, driven by advancements in technologies such as artificial intelligence, natural language

processing, and deep learning. The focus has shifted towards real-time analytics, data governance, privacy concerns, and the ethical use of data.

The history of big data demonstrates the increasing importance of managing and analyzing large and diverse datasets. The field continues to evolve, and organizations are continually exploring innovative approaches and technologies to leverage the potential of big data for insights and decision-making.

IV. Challenges of Big Data

The challenges in **Big Data** are the real implementation hurdles. These require immediate attention and need to be handled because if not handled then the failure of the technology may take place which can also lead to some unpleasant result. Big data challenges include the storing, analyzing the extremely large and fast-growing data.

1. Incomplete Understanding of Big Data:

- Sometimes organizations are unable to understand what is data? Why is it so important? because of this integration of required algorithms to analyze the data becomes really difficult.
- Skilled persons required.
- For example, if employees do not understand the importance of data storage, they might not keep the backup of sensitive data. They might not use databases properly for storage. As a result, when this important data is required, it cannot be retrieved easily

2. Exponential Data Growth:

- One of the biggest challenges is the exponential growth of data as time goes on.
- Since most of the data is unstructured and comes in the form of the document, videos, etc. Which is difficult to analyze.

Since data is growing exponentially, it's becoming difficult to store this data.

3. Security of Data

- Companies spend so many resources for understanding, storing, and analyzing the data they often forget to prioritize the security part.
- So, they try to push the security part for the later stages because of which data breach is possible.

- A solution for these companies is hiring cybersecurity professionals to protect their data.

4. Data Integration

- Since data can be in any form from structured data like phone numbers to unstructured data like videos. So, integrating these data is difficult, but at the same time, data integration is crucial for processes like analysis, reporting, and forecasting.
- So integration has to be error-free. Companies can solve these problems by purchasing the right technologies like IBM Infosphere, Microsoft SQL, etc.

5. Confusion while Big Data tool selection

- Companies often get confused while selecting the best tool for Big Data analysis and storage. Is HBase or Cassandra the best technology for data storage? Is Hadoop MapReduce good enough or will Spark be a better option for data analytics and storage?
- These questions bother companies and sometimes they are unable to find the answers. They end up making poor decisions and selecting inappropriate technology. As a result, money, time, efforts and work hours are wasted.

6. Lack of data professionals

- To run these modern technologies and Big Data tools, companies need skilled data professionals. These professionals will include data scientists, data analysts and data engineers who are experienced in working with the tools and making sense out of huge data sets.

V. Traditional Business Intelligence versus Big Data

Traditional Business Intelligence (BI) :

Traditional Business Intelligence (BI) refers to the methods, technologies, and processes used to collect, analyze, and present structured data from various internal sources within an organization. It involves extracting insights and generating reports to support decision-making processes.

Big Data :

Massive amount of data which cannot be stored, processed and analyzed using traditional tools is known as Big Data. It deals with large volume of both structured, semi structured and unstructured data.

Basis	Business Intelligence (BI)	Big Data
Data Sources	Typically deals with structured data from internal transactional systems, such as databases, spreadsheets, and enterprise resource planning (ERP) systems.	- Involves a wide range of data sources, including structured, unstructured, and semi-structured data from social media, sensors, log files, multimedia content, web interactions, and more. - Big Data often includes data from external sources beyond an organization's traditional data repositories.
Data Volume and Velocity	- Deals with relatively smaller datasets, typically in the gigabyte to terabyte range, and focuses on historical analysis. - Data updates and processing are usually done in batches.	- Handles massive datasets that can range from terabytes to petabytes or even exabytes. - Big Data platforms are designed to handle high-velocity data streams, often requiring real-time or near-real-time processing to extract timely insights
Processing Methods	Relies on structured query language (SQL) and uses pre-defined, structured queries to retrieve and analyze data. It primarily relies on relational databases and data warehouses.	- Utilizes distributed computing frameworks like Apache Hadoop and Apache Spark, which enable parallel processing of large datasets across a cluster of computers. - Big Data technologies support both batch processing and real-time/streaming processing, allowing for more complex and advanced analytics.

Scalability:	• Typically operates on a fixed infrastructure with limited scalability. • It may face challenges when dealing with rapidly growing data volumes or sudden spikes in data processing requirements.	• Offers horizontal scalability, allowing organizations to scale their infrastructure dynamically by adding or removing computing resources based on demand. • Big Data platforms can handle the ever-increasing data volumes and accommodate diverse data sources.
Analytical Capabilities:	• Primarily focuses on descriptive analytics, providing historical insights and reports based on predefined metrics and Key Performance Indicators (KPIs). It often involves simple aggregations, drill-downs, and visualizations	• Enables advanced analytics capabilities, including predictive analytics, machine learning, and data mining. • Big Data platforms leverage complex algorithms and statistical models to discover patterns, correlations, and trends in large and diverse datasets
Decision-Making Speed:	• Offers insights based on historical data and is suited for periodic reporting and decision-making processes with longer cycles.	• Supports real-time or near-real-time analytics, enabling organizations to make faster and more informed decisions based on up-to-date information.

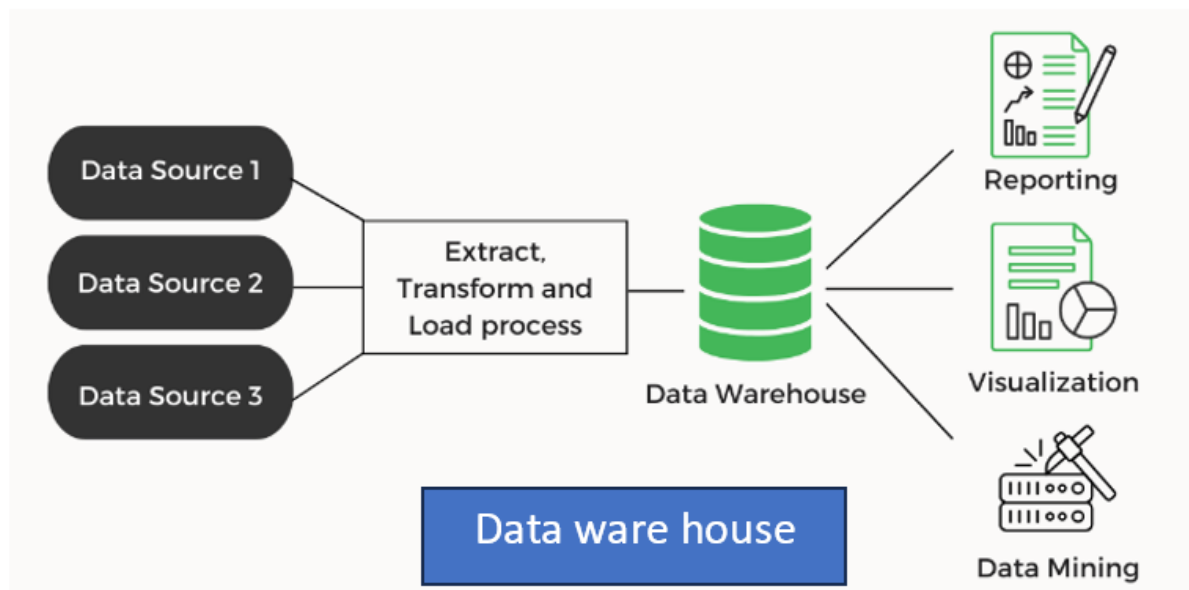
VI. Data Warehouse and Hadoop Environment

A data warehouse and a Hadoop environment are two different concepts and technologies used in managing and processing large amounts of data.

Data Warehouse:

- A data warehouse is a central repository that stores structured, historical data from various sources within an organization.

- It is designed to support business intelligence (BI) and reporting activities



key aspects of a data warehouse:

Structure:

- Data warehouses use a relational database management system (RDBMS) to store data in a structured format.
- Data is organized into tables, rows, and columns, following a predefined schema called a dimensional model.
- The dimensional model typically consists of fact tables (containing numeric measures) and dimension tables (providing context and descriptive attributes).

ETL Processes:

- Extract, Transform, Load (ETL) processes are used to populate the data warehouse.
- Data is extracted from different sources, transformed to adhere to the data warehouse schema, and then loaded into the warehouse.
- ETL processes often involve data cleansing, integration, and aggregation to ensure data quality and consistency.

Historical Data:

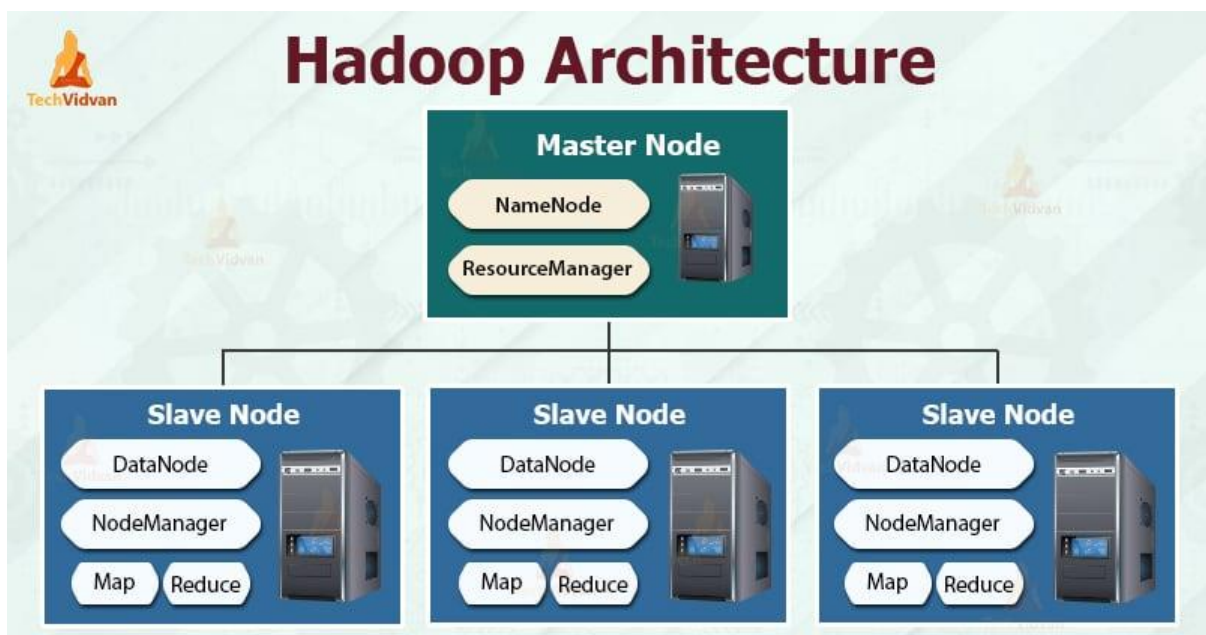
- Data warehouses primarily store historical data, providing a long-term view of the organization's operations.

- Data is typically collected at regular intervals, such as daily, weekly, or monthly, allowing for trend analysis and historical reporting.

Business Intelligence and Reporting:

- Data warehouses serve as a foundation for business intelligence activities.
- They support ad hoc querying, reporting, and analysis by providing a consolidated view of data across different business areas.
- Data is often pre-aggregated and optimized for query performance to facilitate fast and interactive reporting.

Hadoop Environment:



Hadoop is an open-source framework that enables distributed storage and processing of large datasets across clusters of commodity hardware. It provides a scalable and cost-effective solution for managing Big Data.

Distributed Storage:

- Hadoop uses the Hadoop Distributed File System (HDFS) to store data across multiple nodes in a cluster.
- Data is split into blocks and distributed across the cluster, ensuring high availability and fault tolerance.

Distributed Processing:

- Hadoop leverages the MapReduce framework to process data in parallel across the cluster. MapReduce divides data processing tasks into smaller subtasks and distributes them to different nodes, allowing for efficient parallel processing.

Scalability:

- Hadoop is designed to scale horizontally by adding more nodes to the cluster. This enables organizations to store and process large volumes of data without relying on expensive and specialized hardware.

Flexibility and Variety:

- Hadoop can handle various types of data, including structured, unstructured, and semi-structured data. It allows organizations to store and process diverse data formats, such as text, log files, images, videos, and more.

Data Processing Frameworks:

- Hadoop ecosystem includes several data processing frameworks and tools built on top of Hadoop, such as Apache Spark, Apache Hive, and Apache Pig.
- These frameworks provide higher-level abstractions and APIs for data manipulation, querying, and analysis.

Batch and Real-time Processing:

- Hadoop supports both batch processing and real-time/streaming processing.
- While its MapReduce framework is suitable for batch processing, tools like Apache Spark enable real-time and near-real-time analytics on streaming data.

VII. Classification of Data analytics

Big data analytics is process to extract meaningful insights from big data such as hidden patterns, unknown correlation, market trends and customer preferences that can help organizations make informed business decisions.

- Data analytics can be of four types depending on the type and scope of analysis being conducted on the data set.

1. Descriptive analytics

2. Diagnostic analytics

3. Predictive analytics

4. Prescriptive analytics

Descriptive analytics: What happened

- Descriptive analytics uses historical data from a single internal source to describe what happened.
- For example: How many people viewed the website?
- Which products had the most defects?
- Used by most businesses, descriptive analytics forms the crux of everyday reporting, especially through dashboards.

Diagnostic analytics: Why did it happen?

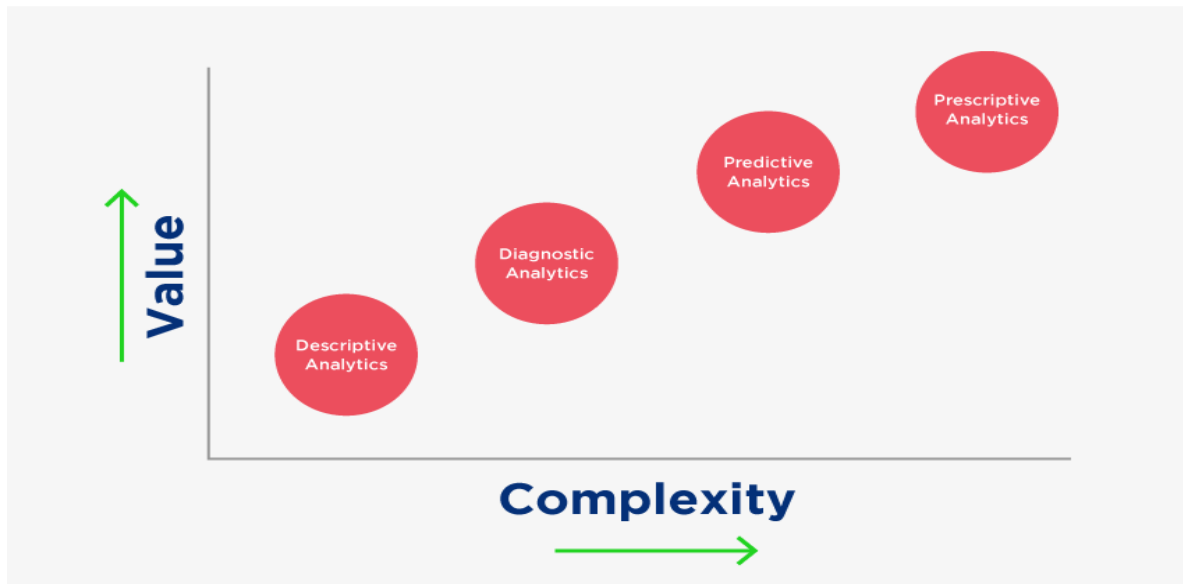
- Diagnostic analytics is a form that dives deep into historical data to identify anomalies, find patterns, identify correlations, and determine causal relationships.
- With the help of diagnostic analysis, data analysts can understand why a certain product did not do well in the market, or why customer satisfaction decreased in a certain month.

Predictive analytics: What might happen next?

- This is a more advanced form of analytics that is often used to answer the question ‘what will happen next?’ in a business situation.
- As the name suggests, this data analytics type predicts the future outcome of a situation depending on all the available data.
- This data will include both market trends and older data about your business performance, By combining these two, this data analytics type can predict how your business will perform during the next season

Prescriptive analytics: What do I need to do?

- Prescriptive analytics is the most complex type of analytics.
- It combines internal data, external sources, and machine-learning techniques to provide the most effective recommendations for business decisions.



VIII. Big data analytics challenges

1. Data quality :

- Poor data quality can significantly impact the accuracy and reliability of analytical results. Incomplete, inconsistent, and inaccurate data can lead to biased or misleading insights.

2. Data Integration:

- Organizations often have data stored in different systems and formats across various departments or sources.
- Integrating data from disparate sources can be complex and time-consuming. Data integration challenges include data harmonization, resolving schema and format differences, and aligning data from different databases or systems..

3. Data Privacy and Security:

- With the increasing focus on data privacy regulations, protecting sensitive and personal data is a significant challenge.

4. Skill Gap and Talent Acquisition:

- There is a shortage of skilled data analysts, data scientists, and data engineers who possess the necessary expertise in data analytics.
- Organizations may struggle to find and retain professionals with the right skill set to drive effective data analytics initiatives.

5. Technology Selection and Integration:

- The data analytics landscape is vast, with numerous tools, platforms, and technologies available.
- Selecting the right technologies that align with organizational needs and integrating them seamlessly with existing systems can be challenging.

6. Scalability and Performance:

- As data volumes and complexity grow, organizations must ensure that their analytics infrastructure can scale accordingly.
- Processing and analyzing large datasets within acceptable timeframes can strain computational resources and affect performance. Designing scalable architectures and optimizing query performance are vital.

7. Technology Selection and Integration:

- The data analytics landscape is vast, with numerous tools, platforms, and technologies available.
- Selecting the right technologies that align with organizational needs and integrating them seamlessly with existing systems can be challenging.

8. Cost Management:

- Implementing and maintaining a robust data analytics infrastructure can be costly.

- Investing in hardware, software, talent, and ongoing maintenance and support can strain budgets.

IX. Top analytical tools

There are several top analytical tools available in the market that are widely used for processing, analyzing, and deriving insights from data. Here are some of the popular analytical tools.

Python:

- Python is a versatile programming language widely used for data analysis and scientific computing.
- It offers numerous libraries and frameworks, such as pandas, NumPy, and scikit-learn, which provide powerful data manipulation, analysis, and machine learning capabilities.

R:

- R is a programming language and environment specifically designed for statistical computing and graphics.
- It offers a wide range of packages and libraries for data manipulation, analysis, and visualization.
- R is popular among statisticians and data scientists for its extensive statistical modeling capabilities.

Apache Spark:

- Apache Spark is an open-source distributed computing system that provides fast and scalable data processing and analytics capabilities.
- It supports various programming languages and offers libraries for distributed data processing, machine learning, and graph analytics.

Apache Hadoop: Apache Hadoop is an open-source framework for distributed processing of large datasets.

- It provides a scalable and fault-tolerant platform for storing, processing, and analyzing big data.
- Hadoop includes components such as Hadoop Distributed File System (HDFS) and MapReduce for distributed data processing.

Cassandra

- [APACHE Cassandra](#) is an open-source NoSQL distributed database that is used to fetch large amounts of data.
- It is capable of delivering thousands of operations every second and can handle petabytes of resources with almost zero downtime.

Mongo DB

- It is a free, open-source platform and a document-oriented (NoSQL) database that is used to store a high volume of data.
- It uses collections and documents for storage and its document consists of key-value pairs which are considered a basic unit of [Mongo DB](#).

It is so popular among developers due to its availability for multi-programming languages such as Python, Jscript, and Ruby

Tableau:

- Tableau is a powerful and user-friendly data visualization and business intelligence tool.
- It allows users to create interactive dashboards, reports, and visualizations from various data sources. Tableau supports advanced analytics, data blending, and provides intuitive drag-and-drop functionality.

Microsoft Power BI:

- Microsoft Power BI is a business intelligence tool that helps users analyze data and share insights.
- It provides interactive dashboards, data visualization, and reporting capabilities.
- Power BI integrates with various data sources, offers AI-powered features, and supports collaboration and data sharing.

SAS:

- SAS (Statistical Analysis System) is a comprehensive suite of analytics tools widely used in industries such as finance, healthcare, and government.
- It provides a wide range of functionalities for data management, advanced analytics, and reporting.

QlikView/Qlik Sense:

- QlikView and Qlik Sense are data visualization and business intelligence tools that enable users to explore and analyze data visually.
- They offer drag-and-drop functionality, interactive dashboards, and powerful data discovery capabilities.

MATLAB:

- MATLAB is a programming and numerical computing environment widely used in academia and industry.
- It offers comprehensive tools for data analysis, modeling, and simulation, along with built-in algorithms and functions for various analytical tasks.

X. Big data analytics important

Big Data Analytics is crucial for several reasons:

Improved Decision Making: Big Data Analytics enables organizations to extract valuable insights from vast and diverse datasets. By analyzing this data, organizations can make data-driven decisions, identify patterns, trends, and correlations that would be otherwise difficult to uncover, and gain a competitive advantage.

Enhanced Operational Efficiency: Big Data Analytics helps optimize operations by providing insights into inefficiencies, bottlenecks, and areas for improvement. By analyzing large datasets, organizations can identify ways to streamline processes, reduce costs, and improve overall operational efficiency.

Personalized Customer Experiences: Analyzing large volumes of customer data allows organizations to understand customer behavior, preferences, and needs on an individual level. With this knowledge, organizations can personalize customer experiences, tailor marketing campaigns, offer personalized recommendations, and enhance customer satisfaction.

Fraud Detection and Security: Big Data Analytics plays a critical role in fraud detection and security. By analyzing large datasets, organizations can identify unusual patterns, detect anomalies, and proactively mitigate risks. This is particularly important in industries such as finance, insurance, and cybersecurity.

Product Development and Innovation: Big Data Analytics enables organizations to gain insights into market trends, customer demands, and emerging patterns. This information is valuable for product development, innovation, and identifying new business opportunities. Organizations can leverage big data to develop new products, improve existing ones, and stay ahead in competitive markets.

Product Development and Innovation: Big Data Analytics enables organizations to gain insights into market trends, customer demands, and emerging patterns. This information is valuable for product development, innovation, and identifying new business opportunities. Organizations can leverage big data to develop new products, improve existing ones, and stay ahead in competitive markets.

Predictive Analytics: Big Data Analytics allows organizations to leverage historical data to build predictive models. These models can forecast future trends, anticipate customer behavior, predict demand, and optimize resource allocation. Predictive analytics helps organizations make proactive decisions and take preventive measures.

Scalability and Agility: Big Data Analytics platforms and technologies are designed to handle large volumes of data, providing the scalability required to process and analyze massive datasets. This enables organizations to adapt quickly to changing business needs and leverage data to gain insights in real-time.

Overall, Big Data Analytics is important as it enables organizations to harness the vast amounts of data available to drive better decision-making, improve operational efficiency, deliver

personalized experiences, detect fraud, fuel innovation, and gain a competitive edge in today's data-driven world.

IMPORTANT QUESTIONS

1. What is Big Data ? Explain Characteristics of Big data?
2. What is big data analytics? Explain five 'V's of Big data. Briefly discuss applications of big data?
3. What are the benefits of Big Data? Discuss challenges under Big Data.?
4. Explain the difference between structure and unstructured data.
5. What is big data analytics and discuss the different types of big data analytics?
6. Differentiate between traditional BI and Big data?
7. Discuss about data ware housing and Hadoop frame work?

UNIT II

INTRODUCTION TO TECHNOLOGY LANDSCAPE

Syllabus :

- 1.NoSQL
2. Comparison of SQL and NoSQL
3. Hadoop -RDBMS Versus Hadoop
4. Distributed Computing Challenges
5. Hadoop Overview - Hadoop Distributed File System - Processing Data with Hadoop
6. Managing Resources and Applications with Hadoop YARN
7. Interacting with Hadoop Ecosystem

I. NoSQL :

NoSQL (Not Only SQL) is a term used to refer to a category of database management systems that provide a non-relational approach to data storage and retrieval. Unlike traditional SQL databases, which are based on the relational model, NoSQL databases offer a flexible schema design, horizontal scalability, and high availability.

NoSQL databases are designed to handle large amounts of unstructured or semi-structured data, which makes them suitable for use cases such as web applications, real-time analytics, content management systems, and distributed systems. They are often used when there is a need for high scalability, low latency, and the ability to handle massive amounts of data

NoSQL databases come in different flavors, each catering to specific requirements:

- a) **Key-value stores:** These databases store data as a collection of key-value pairs. They are simple, highly scalable, and provide fast key-based access. Examples include Redis and Amazon DynamoDB
- b) **Document databases:** Document-oriented databases store and retrieve data in the form of JSON-like documents. They offer flexibility in schema design and support complex nested structures. MongoDB and Couchbase are popular document databases.
- c) **Column-family stores:** These databases organize data into columns grouped by column families or column families. They are well-suited for handling large amounts of structured and semi-structured data. Apache Cassandra and Apache HBase are widely used column-family stores.
- d) **Graph databases:** Graph databases store data in nodes and edges, representing entities and their relationships. They excel at managing highly interconnected data and are used for social networks, recommendation engines, and fraud detection. Examples include Neo4j and Amazon Neptune.

It's important to note that NoSQL databases trade some of the transactional guarantees and data consistency provided by traditional SQL databases for the benefits of scalability and performance. The choice between SQL and NoSQL databases depends on the specific requirements of your application and the nature of the data you need to store and retrieve.

II. Comparison of SQL and NoSQL

The following table highlights the major differences between SQL and NoSQL

Key	SQL	NoSQL
Type	SQL databases are classified as Relational databases, i.e., RDBMS.	NoSQL databases are known as non-relational or distributed database
Language	SQL databases use standard Structured Query Languages, as the name suggests. SQL is an industry-standard and very powerful language to execute complex queries.	NoSQL database has dynamic schema for unstructured data. The data stored in a NoSQL database is not structured. Data could be stored as document-oriented, column oriented, graph-based or organized as a Key-Value store. The syntax can vary from database to database
Scalability	SQL databases can extend their capacity on a single server by increasing their RAM, CPU or SSD. SQL databases are scalable vertically, as their storage could be increased for the same server by enhancing their storage components.	In order to increase the capacity of a NoSQL database, you would have to install new servers parallel to the parent server. NoSQL databases are horizontally scalable which means they can easily handle more traffic by adding new servers to the database, which makes them a great choice for large and constantly changing databases.
Schema	SQL databases have a fixed, pre-defined schema, which makes the data storage more rigid, static, and restrictive.	NoSQL databases don't have a pre-defined schema, which makes them schema-less and more flexible.
Internal implementation	SQL follows ACID (Atomicity, Consistency, Isolation and Durability) properties for its operations.	NoSQL is based on CAP (Consistency, Availability, and Partition Tolerance).

Data Storage	SQL databases can only be run on a single system and because of this, they don't follow the distribution of data and hence they don't support hierarchical storage of data.	NoSQL Databases can run on multiple systems, and hence, they support data distribution features like data repetition, partition, etc., making them the best option for hierarchical storage of data.
Type of Data	SQL databases are table-based databases which makes them better for multi-row transaction applications.	NoSQL is document-based, key-value pair, and graph databases, which makes them better when there are a lot of changes in the data.
Performance and suitability	SQL databases are best suited for complex queries but are not preferred for hierarchical large data storage.	NoSQL databases are not so good for complex queries because these are not as powerful as SQL queries but are best suited for hierarchical large data storage.
Examples	SQL databases are implemented in both open source and commercial databases such as like Postgres & MySQL as open source and Oracle and Sqlite as commercial.	NoSQL is purely open source. Some of its famous implementations are MongoDB, BigTable, Redis, RavenDB, Cassandra, Hbase, Neo4j, and CouchDB.

III. RDBMS VS HADOOP

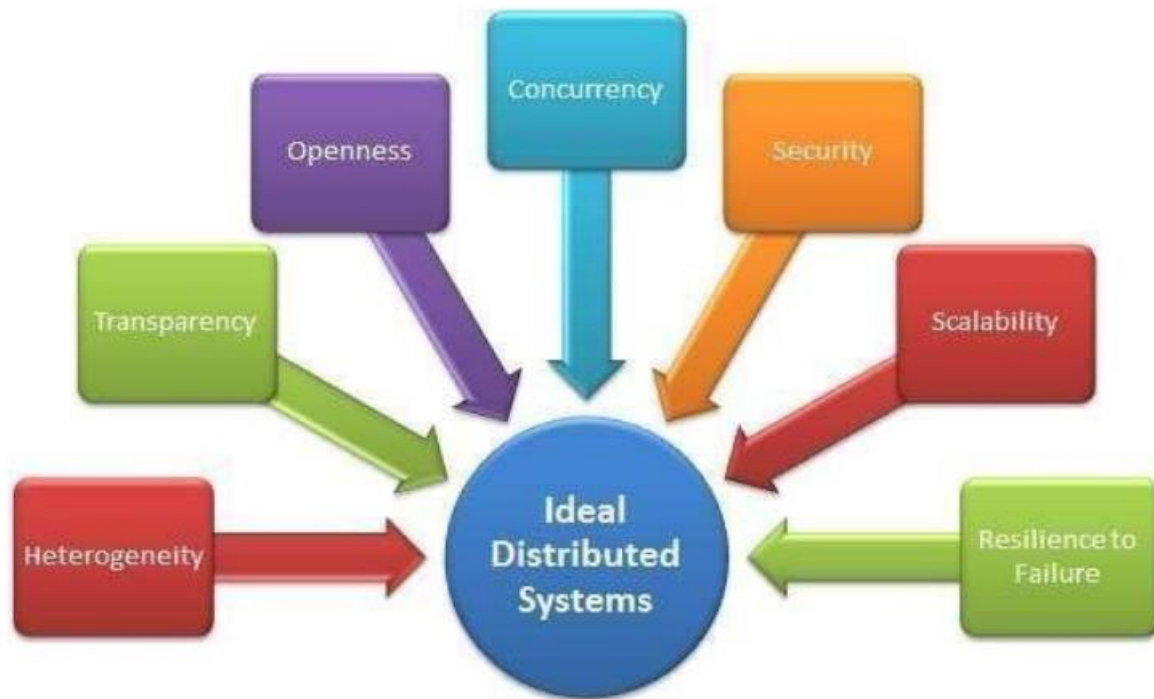
RDBMS (Relational Database Management System) and Hadoop are two distinct technologies used for managing and processing data, and they have different strengths and use cases. Here's a comparison between RDBMS and Hadoop.

Basis	RDBMS	HADOOP
Data variety	Mainly for structured data	Used for structured, unstructured and semi structured data.
Data schema	Required on write (Static schema)	Required on read (Dynamic Schema).

Data storage	Average size data (GBs)	Use for large data sets (Tbs and Pbs)
Querying	SQL language	HQL (Hive Query Language)
Speed	Reads are fast	Both read and write are fast.
Data Processing Model	Primarily suited for Online Transaction Processing (OLTP), where data is structured and accessed for real-time transactions	Suitable for batch processing and complex data analysis tasks like MapReduce, which allows distributed processing of large datasets
Data integrity	High (ACID properties)	Low (CAP properties)
Throughput	High	Low
Data objects	Works on relational tables	Works on Key/value pairs
Use Cases	Best suited for applications with structured data and requirements for data integrity, consistency, and real-time transactions, such as banking systems, CRM applications, and e-commerce platforms	Ideal for big data processing and analysis tasks, like log analysis, data mining, machine learning, recommendation systems, and processing large-scale datasets from various sources.
Data Analysis	Typically uses SQL for querying and analysis, offering strong support for ad-hoc queries, aggregation, and reporting.	Utilizes frameworks like Apache Hive, Pig, and Spark for data processing and analysis, providing powerful tools for complex data transformations and analytics
cost	Cost is applicable for licensed software.	Free of cost, as it is an open source software

IV. Distributed Computing Challenges

Designing a distributed system does not come as easy and straight forward. A number of challenges need to be overcome in order to get the ideal system. The major challenges in distributed systems are listed below



1.Heterogeneity:

The Internet enables users to access services and run applications over a heterogeneous collection of computers and networks. Heterogeneity (that is, variety and difference) applies to all of the following:

- Hardware devices: computers, tablets, mobile phones, embedded devices, etc.
- Operating System: Ms Windows, Linux, Mac, Unix, etc.
- Network: Local network, the Internet, wireless network, satellite links, etc.
- Programming languages: Java, C/C++, Python, PHP, etc.
- Different roles of software developers, designers, system managers.

Different programming languages use different representations for characters and data structures such as arrays and records. These differences must be addressed if programs written in different languages are to be able to communicate with one another. Programs written by different developers cannot communicate with one another unless they use common standards, for example, for network communication.

Heterogeneity and mobile code: The term mobile code is used to refer to program code that can be transferred from one computer to another and run at the destination – Java applets are an example. Code suitable for running on one computer is not necessarily suitable for running on

another because executable programs are normally specific both to the instruction set and to the host operating system.

2. Transparency:

Transparency is defined as the concealment from the user and the application programmer of the separation of components in a distributed system, so that the system is perceived as a whole rather than as a collection of independent components. In other words, distributed systems designers must hide the complexity of the systems as much as they can. Some terms of transparency in distributed systems are

- **Access** Hide differences in data representation and how a resource is accessed
- **Location** Hide where a resource is located
- **Migration** Hide that a resource may move to another location
- **Relocation** Hide that a resource may be moved to another location while in use
- **Replication** Hide that a resource may be copied in several places
- **Concurrency** Hide that a resource may be shared by several competitive users
- **Failure** Hide the
- **failure** and recovery of a resource
- **Persistence** Hide whether a (software) resource is in memory or a disk.

3. Openness

The openness of a computer system is the characteristic that determines whether the system can be extended and re-implemented in various ways. The openness of distributed systems is determined primarily by the degree to which new resource-sharing services can be added and be made available for use by a variety of client programs. If the well-defined interfaces for a system are published, it is easier for developers to add new features or replace sub-systems in the future. Example: Twitter and Facebook have API that allows developers to develop their own software interactively.

4. Concurrency

Both services and applications provide resources that can be shared by clients in a distributed system. There is therefore a possibility that several clients will attempt to access a shared resource at the same time. For example, a data structure that records bids for an auction may be accessed very frequently when it gets close to the deadline time. For an object to be safe in a concurrent environment, its operations must be synchronized in such a way that its data remains

consistent. This can be achieved by standard techniques such as semaphores, which are used in most operating systems.

5. Security Many of the information resources that are made available and maintained in distributed systems have a high intrinsic value to their users. Their security is therefore of considerable importance. Security for information resources has three components: confidentiality (protection against disclosure to unauthorized individuals) integrity (protection against alteration or corruption), availability for the authorized (protection against interference with the means to access the resources).

6. Scalability

Distributed systems must be scalable as the number of users increases. Scalability is defined by B. Clifford Neuman as A system is said to be scalable if it can handle the addition of users and resources without suffering a noticeable loss of performance or increase in administrative complexity. Scalability has 3 dimensions:

- Size o Number of users and resources to be processed. Problem associated is overloading
- Geography o Distance between users and resources. Problem associated is communication reliability o Administration
- As the size of distributed systems increases, many of the system needs to be controlled. Problem associated is administrative mess.

7. Failure Handling

Computer systems sometimes fail. When faults occur in hardware or software, programs may produce incorrect results or may stop before they have completed the intended computation. The handling of failures is particularly difficult.

V. HADOOP OVERVIEW -Processing Data with Hadoop -:

Hadoop is an Apache open source framework written in Java that allows distributed processing of large datasets across clusters of computers using simple programming models. The Hadoop framework application works in an environment that provides distributed storage and computation across clusters of computers. Hadoop is designed to scale up from single server to thousands of machines, each offering local computation and storage.

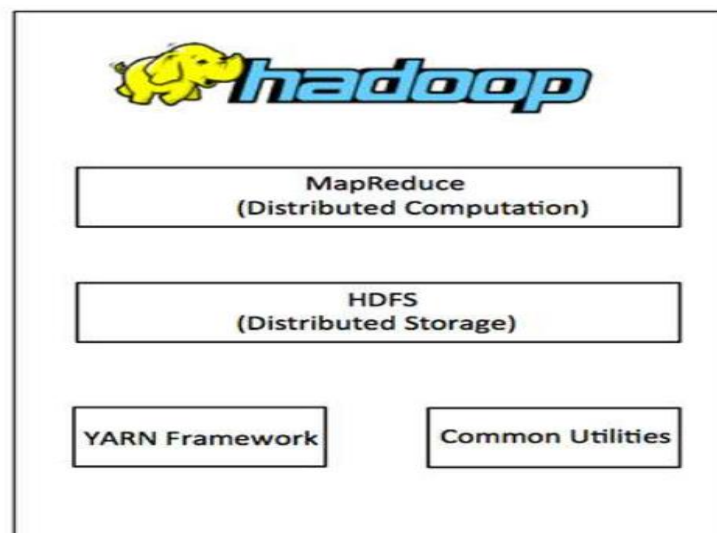
Hadoop Architecture

At its core, Hadoop has two major layers namely –

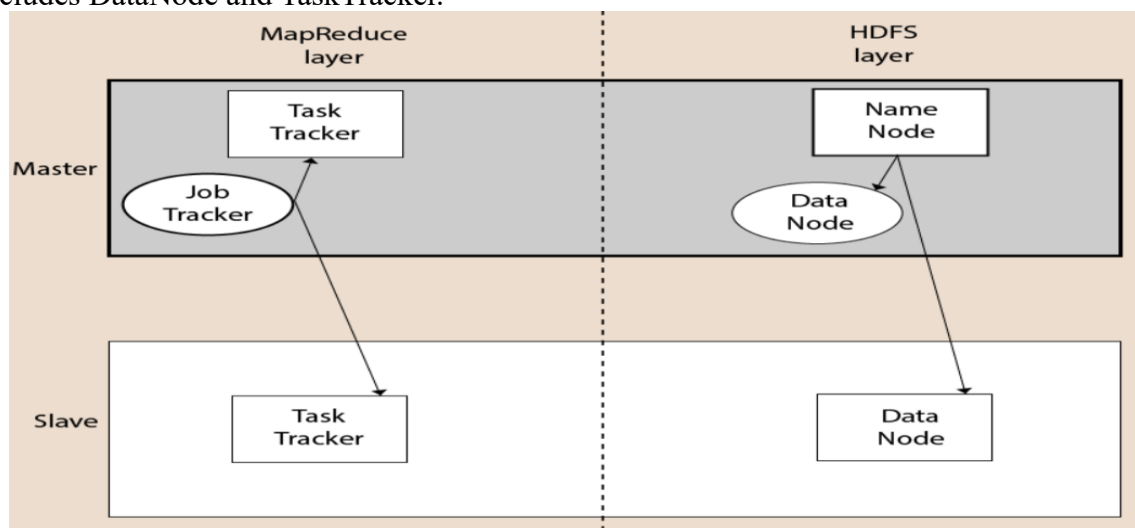
- Processing/Computation layer (**MapReduce**), and
- Storage layer (**Hadoop Distributed File System**).

Apart from the above-mentioned two core components, Hadoop framework also includes the following two modules –

- **Hadoop Common** – These are Java libraries and utilities required by other Hadoop modules.
- **Hadoop YARN** – This is a framework for job scheduling and cluster resource management.



A Hadoop cluster consists of a single master and multiple slave nodes. The master node includes Job Tracker, Task Tracker, NameNode, and DataNode whereas the slave node includes DataNode and TaskTracker.



Hadoop Distributed File System (HDFS):

The Hadoop Distributed File System (HDFS) is a distributed file system for Hadoop. It contains a master/slave architecture. This architecture consists of a single NameNode performing the role of master, and multiple DataNodes performing the role of a slave. Both NameNode and DataNode are capable enough to run on commodity machines.

NameNode

- The NameNode is the most critical component of HDFS. It acts as the master node and is responsible for managing the file system namespace and metadata.
- It keeps track of the location and hierarchy of all files and directories in the HDFS cluster. The metadata includes information such as the location of data blocks, file permissions, and replication factor.
- The NameNode does not store the actual data; instead, it maintains the metadata in memory.

Secondary NameNode :

- Despite its name, the Secondary NameNode is not a standby NameNode or a backup. Instead, it assists the primary NameNode by periodically merging and checkpointing the file system metadata.
- It takes snapshots of the metadata from the primary NameNode and combines them with the edits log to create a new checkpoint. This process helps the NameNode recover more quickly in the event of a failure.

fsimage : the fsimage (file system image) is a snapshot of the file system namespace and metadata at a specific point in time. It contains information about the directory structure, file names, permissions, file sizes, block locations, and replication factors.

Edit Log : The edit log (also known as the transaction log) is a dynamic log that records all the changes (edits) made to the HDFS metadata since the last fsimage checkpoint. These changes include file creations, deletions, updates, permission changes, and other file system operations.

DataNode

- DataNodes are worker nodes in the HDFS cluster responsible for storing the actual data blocks. They are managed by the NameNode and store data locally on their disks.
- DataNodes report their status and the list of data blocks they are hosting to the NameNode periodically. The NameNode uses this information to ensure data replication and data integrity

Job Tracker

- The JobTracker is the central coordinating daemon for MapReduce jobs in Hadoop 1.x versions. In Hadoop 2.x and later, it has been replaced by the ResourceManager in the YARN (Yet Another Resource Negotiator) framework.
- The JobTracker is a master node that manages the entire MapReduce job lifecycle. It receives job submissions from clients, breaks down the job into smaller tasks (map tasks and reduce tasks), and schedules those tasks on available TaskTracker nodes in the cluster.

Task Tracker

- The TaskTracker runs on each data node (worker node) in the Hadoop cluster. It is responsible for executing individual map and reduce tasks as directed by the JobTracker (or ResourceManager in Hadoop 2.x).
- When a TaskTracker starts up, it communicates with the JobTracker (or ResourceManager) to indicate its availability and the number of available processing slots (known as task slots).

the JobTracker (replaced by ResourceManager in Hadoop 2.x) is the master node responsible for managing and coordinating MapReduce jobs, while the TaskTracker (running on data nodes) executes individual tasks and reports their status back to the JobTracker (or ResourceManager). The interaction between these components enables the distributed processing of MapReduce jobs on a Hadoop cluster.

MapReduce:

MapReduce is a programming model and processing paradigm developed by Google that simplifies the processing of large-scale data across distributed computing clusters. It is widely used in Apache Hadoop, an open-source framework for distributed storage and processing of big data.

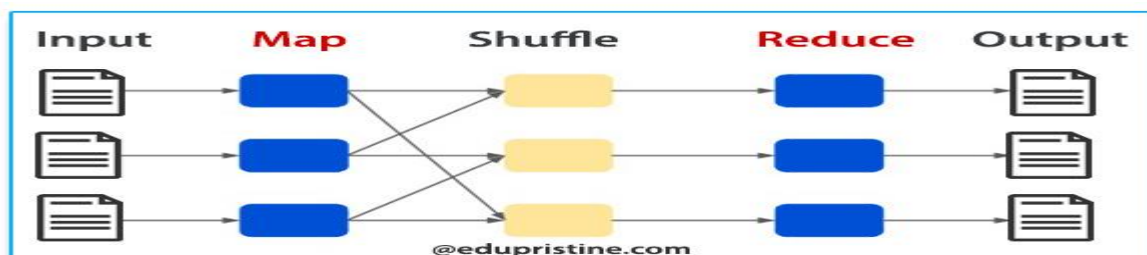
The MapReduce paradigm consists of two main phases:

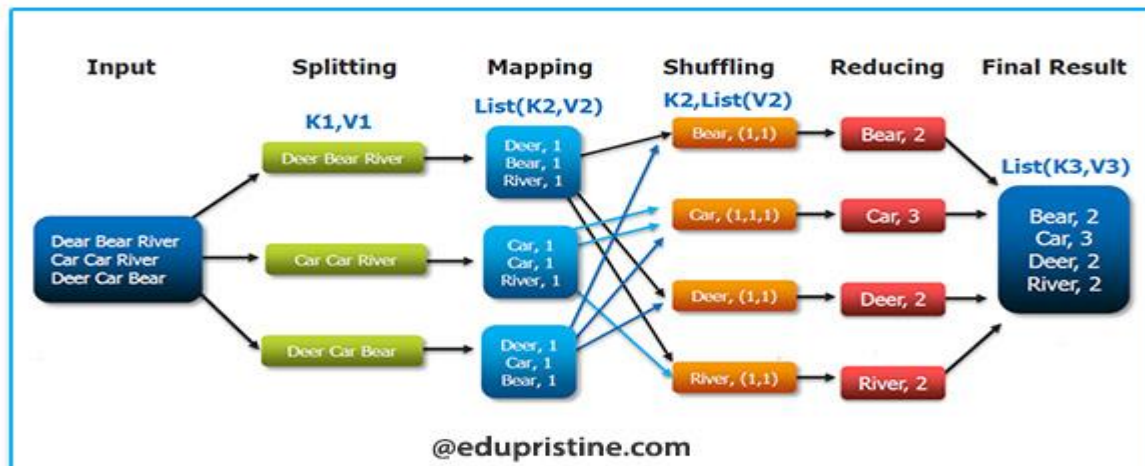
1. Map Phase

- During the map phase, the input data is divided into smaller chunks, known as input splits. Each input split is processed independently by a mapper task.
- The mapper task takes each input split and applies a user-defined Map function to it. The Map function transforms the input data into key-value pairs, where the key represents a unique identifier, and the value contains the data related to that key.
- The output of the Map function is an intermediate set of key-value pairs, known as intermediate key-value pair.

2. Reduce Phase

- After the map phase is completed, the intermediate key-value pairs are shuffled and sorted based on their keys. This process groups all values associated with the same key together, allowing the reduce phase to process them efficiently.
- The sorted intermediate key-value pairs are passed to the reducer tasks, which execute a user-defined Reduce function.
- The Reduce function takes the sorted key-value pairs as input and performs aggregation or computation on the values associated with each key.
- the output of the Reduce function is a final set of key-value pairs, which represents the processed and aggregated results of the MapReduce job.





Hadoop Working

Hadoop runs code across a cluster of computers. This process includes the following core tasks that Hadoop performs –

- Data is initially divided into directories and files. Files are divided into uniform sized blocks of 128M and 64M (preferably 128M).
- These files are then distributed across various cluster nodes for further processing.
- HDFS, being on top of the local file system, supervises the processing.
- Blocks are replicated for handling hardware failure
- Checking that the code was executed successfully.
- Performing the sort that takes place between the map and reduce stages.
- Sending the sorted data to a certain computer.
- Writing the debugging logs for each job.

Advantages of Hadoop

- Hadoop framework allows the user to quickly write and test distributed systems. It is efficient, and it automatically distributes the data and work across the machines and in turn, utilizes the underlying parallelism of the CPU cores.
- Hadoop does not rely on hardware to provide fault-tolerance and high availability (FTHA), rather Hadoop library itself has been designed to detect and handle failures at the application layer.
- Servers can be added or removed from the cluster dynamically and Hadoop continues to operate without interruption.
- Another big advantage of Hadoop is that apart from being open source, it is compatible on all the platforms since it is Java based.
-

VI. Managing Resources and Applications with Hadoop YARN

YARN, known as Yet Another Resource Negotiator, was introduced in Hadoop version 2.0 by Yahoo and Hortonworks in 2012. The basic idea of YARN in Hadoop is to divide the functions of resource management and task scheduling/monitoring into separate daemon processes.

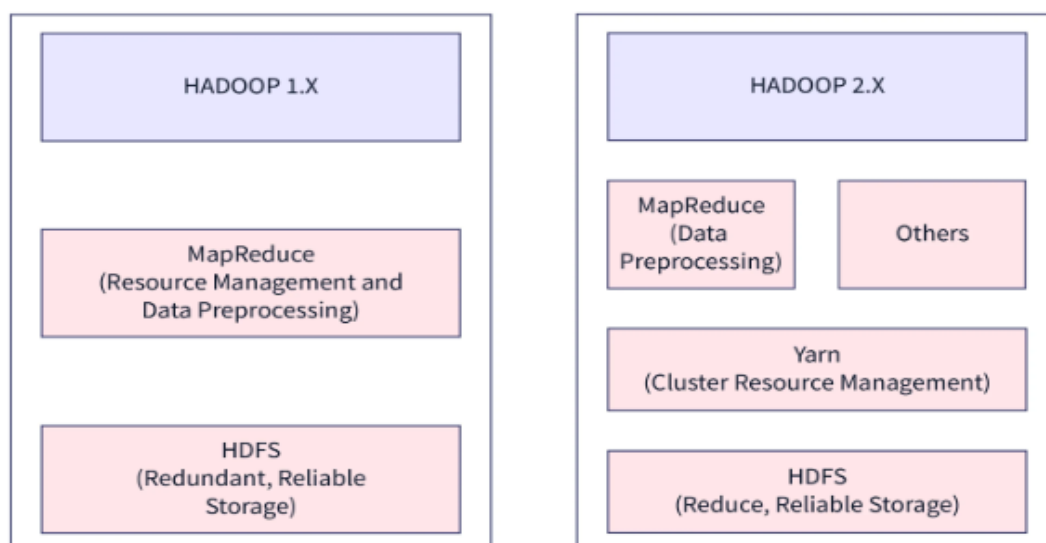
YARN in Hadoop allows for the execution of various data processing engines such as batch processing, graph processing, stream processing, and interactive processing, as well as the processing of data stored in HDFS.

Why is YARN Hadoop Used?

Before Hadoop 2.0, Hadoop 1.x had two main components: the Hadoop Distributed File System (HDFS) and MapReduce. The MapReduce batch processing framework was tightly coupled to HDFS.

By relying solely on MapReduce, Hadoop ran into many challenges.

- MapReduce had to handle both resource management and processing.
- Job Tracker was overloaded with many features that it had to manage, including planning, task control, processing, resource allocation,
- One Job Tracker was the bottleneck in terms of scalability
- Overall, the system was computationally inefficient in terms of resource usage.
- Systems with Hadoop 1.0 can only run MapReduce applications.



How does Apache Hadoop YARN Work?

The basic idea of YARN in Hadoop is to separate the two main responsibilities of JobTracker and TaskTracker into separate entities.

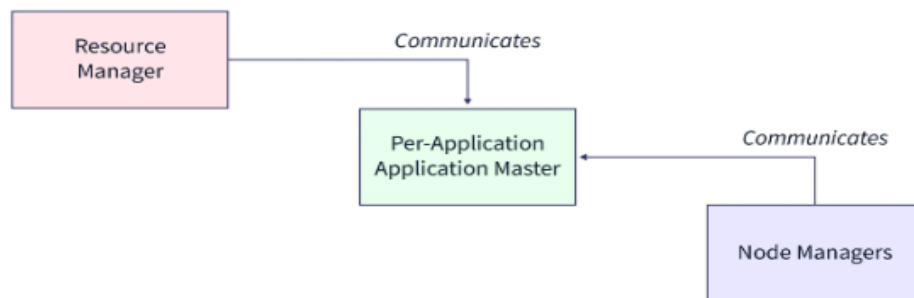
YARN consists of the following components:

Resource Manager :

The resource manager's responsibility is to allocate available resources to applications

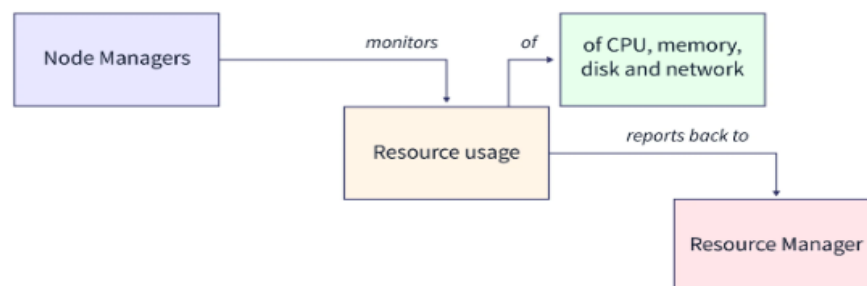
Per-application ApplicationMaster:

On one side, the ApplicationMaster communicates with Resource Manager and on the other side with Node Manager. It negotiates resources from Resource Manager and works with the Node Manager to execute and monitor the component tasks.



Node Manager and Container :

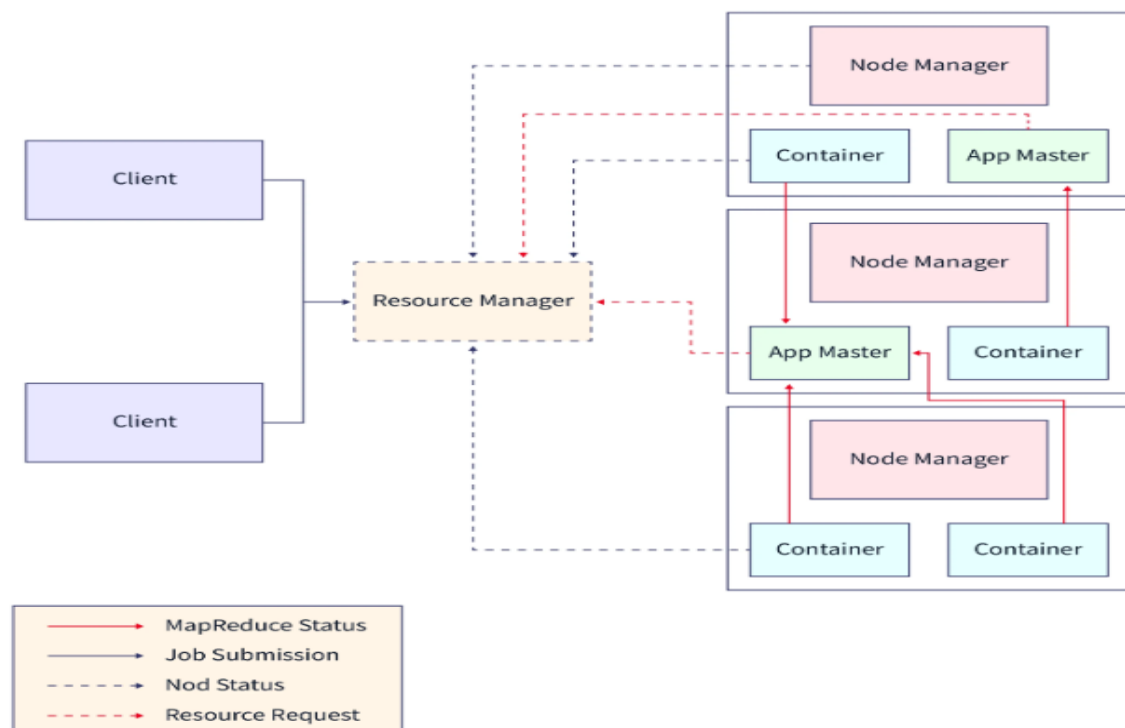
Node Manager is a slave that runs on each computer and is in charge of launching the application's containers.



YARN Architecture in Hadoop

The architecture of YARN is shown in the figure below. The architecture consists of several components such as Resource Manager, Node Manager and Application Master.

The cluster's Resource Manager and Node Manager are two components in charge of managing and scheduling Hadoop jobs. The execution of tasks in parallel is the responsibility of the Application Master. Its daemon is in charge of carrying out the compute jobs, checking them for errors, and finishing them.



Main Components of YARN Architecture in Hadoop

Resource Manager

The Resource Manager is the central decision maker for allocating resources among all system applications. When it receives processing requests, it forwards portions of them to the appropriate node managers, where the actual processing takes place. It acts as the cluster's resource arbitrator, allocating available resources to competing applications.

The Resource Manager consists of the following :

1. Scheduler
2. Application Manager

- **Scheduler** : It is known as a pure scheduler as it performs monitoring or application state tracking.
- If there is a sudden failure in hardware or application failure, it does not guarantee a restart of the failed tasks.
- The scheduler performs its functions based on the resource requirements of the application. It does this by using the abstraction of resource containers, including memory, CPU, disk, network, and more
- **Application Manager**: The Application Manager is responsible for collecting job submissions, selecting the first container to run the application-specific ApplicationMaster, and providing services to restart the ApplicationMaster container in case of failure.
- Each application's ApplicationMaster is responsible for negotiating the appropriate resource containers from the scheduler, maintaining their state and tracking progress

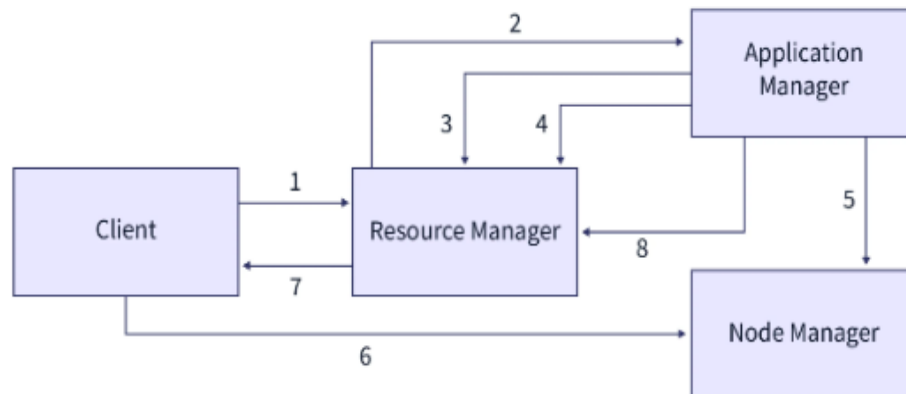
Node Manager

- Monitors resource usage (storage, CPU, etc) per container and handles log management.
- It registers with the Resource Manager and sends out heartbeats containing the health status of the node.
- Resource Manager assigns the Node Manager to manage all the application containers

Application Master

- A resource request for a container to perform an application task is sent from the application host to the Resource Manager
- Upon receiving the request from the Application Master, the Resource Manager evaluates resource requirements, checks resource availability, and authorizes the container to fulfil the resource request.
- Once the container is configured, the application host will instruct the Node Manager to use resources and start application-specific activities and also sends health reports to the Resource Manager from time to time.

Application Workflow in Hadoop YARN



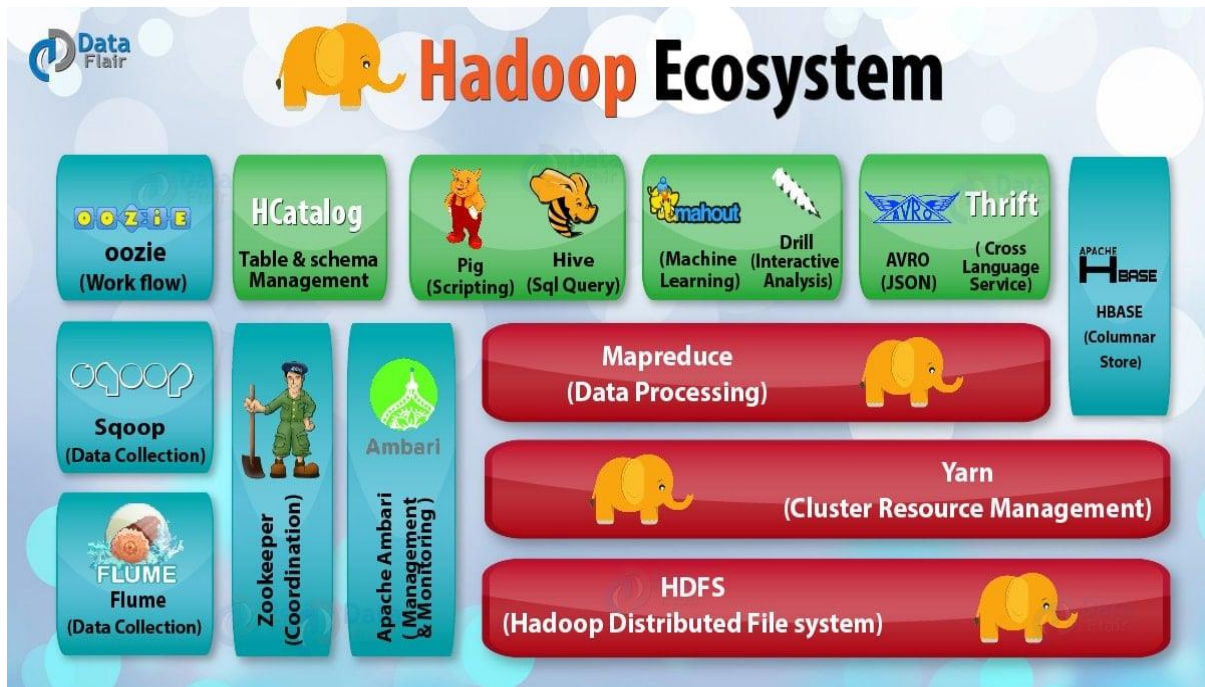
1. A client applies.
2. To launch the Application Master, the Resource Manager allows a container.
3. The Resource Manager accepts the Application Master registration.
4. Containers are negotiated by the Application Master with the Resource Manager.
5. The Node Manager receives a request to launch containers from the Application Master.
6. The container is used to run application code.
7. To check on the status of an application, the client contacts the Resource Manager or Application Master.
8. The Application Master deregisters with the Resource Manager once the above processing is finished.

VII. Hadoop Ecosystem

Apache Hadoop ecosystem refers to the various components of the Apache Hadoop software library; it includes open source projects as well as a complete range of complementary tools.

Some of the most well-known tools of the Hadoop ecosystem include:

1. HDFS
2. MapReduce
3. YARN
4. Sqoop
5. Flame
6. Hive
7. Pig
8. Hbase
9. Oozie,
10. Zookeeper, etc.



1. HDFS:

- HDFS is the primary or major component of Hadoop ecosystem and is responsible for storing large data sets of structured or unstructured data across various nodes and thereby maintaining the metadata in the form of log files.
- HDFS consists of two core components i.e.
 - Name node
 - Data Node
 - Name Node is the prime node which contains metadata (data about data) requiring comparatively fewer resources than the data nodes that stores the actual data.
 - HDFS maintains all the coordination between the clusters and hardware, thus working at the heart of the system.

2.YARN

- Yet Another Resource Negotiator, as the name implies, YARN is the one who helps to manage the resources across the clusters. In short, it performs scheduling and resource allocation for the Hadoop System.

Consists of three major components i.e.

- Resource Manager
- Nodes Manager
- Application Manager

3.MapReduce:

- By making the use of distributed and parallel algorithms, MapReduce makes it possible to carry over the processing's logic and helps to write applications which transform big data sets into a manageable one.
- MapReduce makes the use of two functions i.e. Map() and Reduce() whose task is:
 - Map() performs sorting and filtering of data and thereby organizing them in the form of group. Map generates a key-value pair based result which is later on processed by the Reduce() method.
 - Reduce(), as the name suggests does the summarization by aggregating the mapped data. In simple, Reduce() takes the output generated by Map() as input and combines those tuples into smaller set of tuples.

4.HBase

- Apache HBase is a Hadoop ecosystem component which is a distributed database that was designed to store structured data in tables that could have billions of row and millions of columns

5.Sqoop

- imports data from external sources into related Hadoop ecosystem components like HDFS, Hbase or Hive.
- It also exports data from Hadoop to other external sources. Sqoop works with relational databases such as teradata, Netezza, oracle, MySQL

6.Flume

- Efficiently collects non relational data base, aggregate and moves a large amount of data from its origin and sending it back to HDFS.

7.PIG:

- This is a high-level scripting language used to execute queries for larger datasets that are used within Hadoop.
- Pig's simple SQL-like scripting language is known as Pig Latin and its main objective is to perform the required operations and arrange the final output in the desired format.

8.HIVE:

- Hive is an ETL and Data warehousing tool used to query or analyze large datasets stored within the Hadoop ecosystem. Hive has three main functions: data summarization, query, and analysis of unstructured and semi-structured data in Hadoop.
- With the help of SQL methodology and interface, HIVE performs reading and writing of large data sets. However, its query language is called as HQL (Hive Query Language).

9.Apache Mahout :

- Mahout is open source framework for creating scalable [machine learning](#) algorithm and data mining library.
- Once data is stored in Hadoop HDFS, mahout provides the data science tools to automatically find meaningful patterns in those big data sets.

10.Apache Zookeeper

- Zookeeper manages and coordinates a large cluster of machines.

11.Oozie

- It is a workflow scheduler system for managing Apache Hadoop jobs. Oozie combines multiple jobs sequentially into one logical unit of work.

UNIT III

INTRODUCTION TO MONGODB AND MAPREDUCE PROGRAMMING

Syllabus :

1. MongoDB: Why Mongo DB
2. Terms used in RDBMS and Mongo DB
3. Data Types
4. MongoDB Query Language.
5. MapReduce: Mapper – Reducer – Combiner – Partitioner – Searching – Sorting – Compression.

I. **MongoDB :**

MongoDB is a popular open-source, document oriented NoSQL (not only SQL) database management system designed for scalability, flexibility, and high-performance.

Why Mongo DB?

Key features of MongoDB include:

- **Document-oriented:** MongoDB stores data in flexible, self-describing documents using a format called BSON (Binary JSON). Documents can have varying structures, allowing for easy and dynamic schema evolution.
- **Scalability and Performance:** MongoDB is designed to scale horizontally across multiple servers, enabling it to handle large volumes of data and high traffic loads. It provides automatic sharding and replication capabilities for distributing data and ensuring fault tolerance.
- **Querying and Indexing:** MongoDB supports rich query capabilities, including ad-hoc queries, indexing, and aggregation pipelines. It has a flexible query language that allows for complex searches and supports various types of indexes for optimizing query performance.

- **High Availability and Fault Tolerance:** MongoDB provides built-in replication, which allows data to be automatically synchronized across multiple servers, ensuring data redundancy and high availability. In case of server failures, MongoDB can seamlessly failover to a replica set member.
- **Flexible Data Model:** MongoDB's document-oriented data model allows for flexible and dynamic schemas, accommodating evolving data structures without requiring downtime or complex migrations. It is well-suited for scenarios where data has varied attributes or evolving requirements.
- **Integration and Ecosystem:** MongoDB has a rich ecosystem with support for various programming languages and frameworks. It provides drivers and connectors for popular programming languages, making it easy to integrate MongoDB with applications.

II. Terms used in RDBMS and Mongo DB

	RDBMS	MongoDB
Database	DATABASE	DATABASE
Table vs. Collection	In an RDBMS, data is organized in tables , which consist of rows and columns	In MongoDB, data is stored in collections , which are analogous to tables in RDBMS.
Row vs. Document:	Each row in an RDBMS table represents a single record with a fixed set of columns and data types	In MongoDB, each document in a collection is a JSON-like object with a dynamic schema, allowing for varying fields within the same collection.
Column vs. Field	Columns represent the individual data attributes or fields of a table	Fields are the equivalent data attributes of a document in a collection

JOIN	A JOIN operation combines rows from two or more tables based on a related column between them	MongoDB does not support JOINS directly since it follows a denormalized data model. Instead, data is often embedded within documents to avoid JOIN operations
Primary Key vs. _id:	A primary key is a unique identifier for each row in a table, used for fast retrieval and data integrity.	Each document in a collection is assigned a unique identifier called _id , which acts as a primary key.
Normalization vs. Denormalization:	Normalization is a process of structuring data to eliminate redundancy and improve data integrity. This is achieved by dividing data into multiple related tables	Denormalization is often used in MongoDB to improve query performance by embedding related data within a single document, reducing the need for multiple queries.
ACID Transactions	RDBMS supports ACID (Atomicity, Consistency, Isolation, Durability) transactions, ensuring data integrity and consistency.	MongoDB offers ACID transactions for specific operations within a single document or across documents in a single collection
Index:	An index is used to speed up data retrieval in an RDBMS by creating a separate data structure that allows for faster lookup based on indexed columns.	MongoDB also supports indexing for faster queries on specific fields within a collection.

MongoDB Query Language

What is MongoDB Query?

- MongoDB Query is a way to get the data from the MongoDB database. MongoDB queries provide the simplicity in process of fetching data from the database, it's similar to SQL queries in SQL Database language.
- While performing a query operation, one can also use criteria or conditions which can be used to retrieve specific data from the database.
- MongoDB stores the data in the form of the structure **(field:value pair)** rather than tabular form.
- It stores data in BSON (Binary JSON) format just like JSON format.
- A simple example of a MongoDB database collection

```
{
  "_id" : ObjectId("6009585d35cce6b7b8f087f1"),
  "title" : "Math",
  "author" : "Aditya",
  "level" : "basic",
  "length" : 230,
  "example" : 11
}
```

example

- Assume we have created a collection named mycol as –

```
< use sampleDB
```

```
<switched to db sampleDB
```

```
< db.createCollection("mycol")
```

```
{ "ok" : 1 }
```

```
>
```

- And inserted 3 documents in it using the insert() method as shown below –

```
> db.mycol.insert([
  {
    title: "MongoDB Overview",
    description: "MongoDB is no SQL database",
    by: "tutorials point",
    url: "http://www.tutorialspoint.com",
    tags: ["mongodb", "database", "NoSQL"],
    likes: 100
  },
```

```

  {
    title: "NoSQL Database",
    description: "NoSQL database doesn't have tables",
    by: "tutorials point",
    url: "http://www.tutorialspoint.com",
    tags: ["mongodb", "database", "NoSQL"],
    likes: 20,
    comments: [
      {
        user: "user1",
        message: "My first comment",
        dateCreated: new Date(2013,11,10,2,35),
        like: 0
      }
    ]
  }
])
```

MongoDB uses a flexible and powerful query language to interact with its databases. The MongoDB query language allows you to retrieve, insert, update, and delete data from the database. Here are some common elements of the MongoDB query language:

1. Find:

The `find()` method is used to retrieve data from a collection based on specified criteria.

You can specify the filter conditions as a JSON object in the `find()` method to retrieve documents that match the criteria.

Syntax

- The basic syntax of **find()** method is as follows-

```
> db.COLLECTION_NAME.find()
```

find() method will display all the documents in a non-structured way.

- Following method retrieves all the documents in the collection –
- ```
> db.mycol.find()
```
- ```
{ "_id" : ObjectId("5dd4e2cc0821d3b44607534c"), "title" : "MongoDB Overview",  
  "description" : "MongoDB is no SQL database", "by" : "tutorials point", "url" :  
  "http://www.tutorialspoint.com", "tags" : [ "mongodb", "database", "NoSQL" ], "likes"  
  : 100 } { "_id" : ObjectId("5dd4e2cc0821d3b44607534d"), "title" : "NoSQL Database",  
  "description" : "NoSQL database doesn't have tables", "by" : "tutorials point", "url" :  
  "http://www.tutorialspoint.com", "tags" : [ "mongodb", "database", "NoSQL" ], "likes"  
  : 20, "comments" : [ { "user" : "user1", "message" : "My first comment", "dateCreated"  
    : ISODate("2013-12-09T21:05:00Z"), "like" : 0 } ] } >
```

pretty()

To display the results in a formatted way, you can use `pretty()` method.

Syntax

```
>db.COLLECTION_NAME.find().pretty()
```

- ```
> db.mycol.find().pretty()
```

```
{
 "_id" : ObjectId("5dd4e2cc0821d3b44607534c"),
 "title" : "MongoDB Overview",
 "description" : "MongoDB is no SQL database",
 "by" : "tutorials point",
 "url" : "http://www.tutorialspoint.com",
 "tags" : [
 "mongodb",
 "database",
 "NoSQL"
],
 "likes" : 100
}
```

```

{
 "_id" : ObjectId("5dd4e2cc0821d3b44607534d"),
 "title" : "NoSQL Database",
 "description" : "NoSQL database doesn't have tables",
 "by" : "tutorials point",
 "url" : "http://www.tutorialspoint.com",
 "tags" : [
 "mongodb",
 "database",
 "NoSQL"
],
 "likes" : 20,
 "comments" : [
 {
 "user" : "user1",
 "message" : "My first comment",
 "dateCreated" : ISODate("2013-12-09T21:05:00Z"),
 "like" : 0
 }
]
}

```

## findOne()

- Apart from the find() method, there is findOne() method, that returns only one document.

## Syntax

- >db.COLLECTIONNAME.findOne()

```

> db.mycol.findOne({title: "MongoDB Overview"})
{
 "_id" : ObjectId("5dd6542170fb13eec3963bf0"),
 "title" : "MongoDB Overview",
 "description" : "MongoDB is no SQL database",
 "by" : "tutorials point",
 "url" : "http://www.tutorialspoint.com",
 "tags" : [
 "mongodb",
 "database",
 "NoSQL"
],
 "likes" : 100
}

```

## Equal filter query

- The equality operator(\$eq) is used to match the documents where the value of the field is equal to the specified value. In other words, the \$eq operator is used to specify the equality condition.

### Syntax:

➤ `db.collection_name.find(<key> : {$eq : <value>})`

Example :

- `>db.article.find({author: {$eq:"devil"}}).pretty()`

## Comparison Operators:

MongoDB supports various comparison operators such as \$eq, \$ne, \$gt, \$lt, \$gte, and \$lte to compare values in queries.

### Syntax:

```
db.collection_name.find(<key> : {$gte : <value>})
or
db.collection_name.find(<key> : {$lte : <value>})
```

### Example:

```
db.article.find({length: {$gte: 510}}).pretty()
```

## Logical Operators:

MongoDB provides logical operators like \$and, \$or, and \$not to combine multiple conditions in a query.

## AND in MongoDB

In MongoDB, the AND logical operator is used to combine multiple conditions in a query to retrieve documents that satisfy all of the specified conditions simultaneously

## Syntax

```
>db.mycol.find({ $and: [{<key1>:<value1>}, { <key2>:<value2>}] })
```

Following example will show all the tutorials written by 'tutorials point' and whose title is 'MongoDB Overview'

```
< db.mycol.find({$and:[{"by":"tutorials point"}, {"title": "MongoDB Overview"}]}).pretty()
```

## OR in MongoDB

In MongoDB, the OR logical operator is used to combine multiple conditions in a query to retrieve documents that satisfy at least one of the specified conditions.

## Syntax

- ```
>db.mycol.find( { $or: [ {key1: value1}, {key2:value2} ] } ).pretty()
```

Following example will show all the tutorials written by 'tutorials point' or whose title is 'MongoDB Overview'.

```
>db.mycol.find({$or:[{"by":"tutorials point"}, {"title": "MongoDB Overview"}]}).pretty()
```

NOT in MongoDB

syntax

```
>db.COLLECTION_NAME.find( { $not: [ {key1: value1}, {key2:value2} ] } ).pretty()
```

Following example will retrieve the document(s) whose age is not greater than 25

```
> db.empDetails.find( { "Age": { $not: { $gt: "25" } } } )
```

NOR operator

In MongoDB, the NOR logical operator is used to combine multiple conditions in a query to retrieve documents that do not satisfy any of the specified conditions.

Syntax

```
>db.collection.find({ $nor: [ { condition1 }, { condition2 }, // Add more conditions as needed ] })
```

Example

- Now, let's say we want to find products that do not have the name "Product A" and whose price is not less than 20. We can use the \$nor javascript operator as follows:

```
>db.products.find({ $nor: [ { name: "Product A" }, { price: { $lt: 20 } } ]})
```

Projection:

The **project()** method is used to specify which fields to include or exclude from the query result.

Sorting:

The **sort()** method is used to sort the query result based on one or more fields.

Limiting:

The **limit()** method is used to restrict the number of documents returned by a query.

Aggregation:

MongoDB supports the Aggregation Framework, which allows you to perform advanced data processing and transformation tasks, including grouping, filtering, and computing aggregate functions.

Insert:

To insert data into a collection, you use the **insertOne()** or **insertMany()** method.

Update:

The **updateOne()** and **updateMany()** methods are used to update existing documents in a collection based on specified criteria.

Delete:

To delete documents from a collection, you can use the **deleteOne()** or **deleteMany()** method.

The MongoDB query language is designed to be intuitive and easy to use. It provides a wide range of capabilities for retrieving and manipulating data, making it suitable for various use cases, including real-time applications, analytics, and big data processing. MongoDB's query language allows developers to efficiently work with both simple and complex data structures, providing the flexibility needed for modern data-driven applications.

RDBMS Vs MongoDB

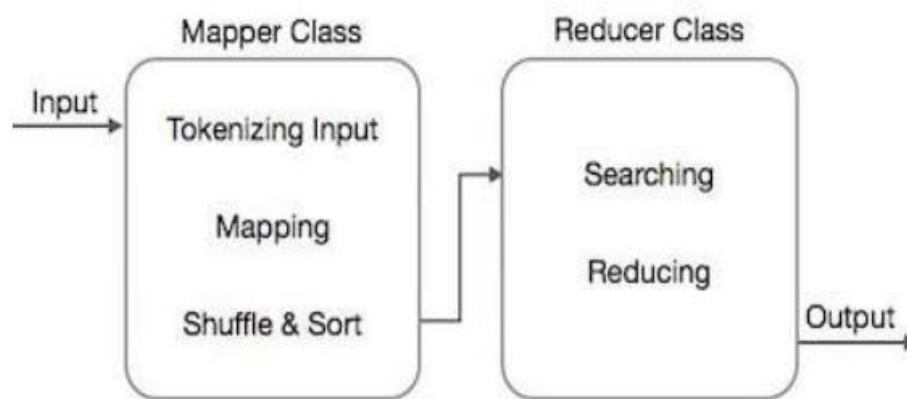
	RDBMS	MongoDB
Retrieving Data	SELECT column1, column2 FROM table_name WHERE condition;	db.collection_name.find({ field1: value1, field2: value2 }, { field1: 1, field2: 1 });
Inserting Data	INSERT INTO table_name (column1, column2) VALUES (value1, value2);	db.collection_name.insertOne({ field1: value1, field2: value2 });
Updating Data:	UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;	db.collection_name.updateOne({ field: value }, { \$set: { field1: value1, field2: value2 } });
Deleting Data:	DELETE FROM table_name WHERE condition;	db.collection_name.deleteOne({ field: value });
Aggregation (Grouping and Aggregating Data)	SELECT column1, COUNT(column2) FROM table_name GROUP BY column1;	db.collection_name.aggregate([{ \$group: { _id: "\$field1", count: { \$sum: 1 } } }, { \$project: { _id: 0, field1: "\$_id", count: 1 } }]);
Sorting:	SELECT column1, column2 FROM table_name ORDER BY column1 DESC, column2 ASC;	db.collection_name.find({}).sort({ field1: -1, field2: 1 });
Limiting:	SELECT column1, column2 FROM table_name LIMIT 10;	db.collection_name.find({}).limit(10);

MapReduce: Mapper – Reducer – Combiner – Partitioner – Searching – Sorting – Compression

The MapReduce algorithm contains two important tasks, namely Map and Reduce.

- The map task is done by means of the Mapper Class
- The reduce task is done by means of Reducer Class.

Mapper class takes the input, tokenizes it, maps and sorts it. The output of Mapper class is used as input by Reducer class, which in turn searches matching pairs and reduces them.



MapReduce implements various mathematical algorithms to divide a task into small parts and assign them to multiple systems. In technical terms, MapReduce algorithm helps in sending the Map & Reduce tasks to appropriate servers in a cluster.

Mapper:

Input: The input data is divided into smaller chunks called input splits.

Mapper Function: The Mapper is responsible for processing these input splits independently. It applies a user-defined function to each input split to produce a set of intermediate key-value pairs. The key is typically used for grouping and sorting in the next phase (Shuffle and sort).

Combiner:

Optional: The Combiner is an optional intermediate step that can be used to reduce the volume of data transferred between the Mapper and Reducer phases. It performs a local aggregation of the output from Mappers on each node before sending it to the Reducer. This is particularly useful when the same key appears multiple times in the Mapper output.

The primary purpose of the combiner is to reduce the volume of the data that needs to be transferred between the mapper and Reducer and improve the overall efficiency of the MapReduce job.

Partitioner:

Partitioning Logic: The Partitioner decides which Reducer will receive each intermediate key-value pair from the Mapper. It is responsible for ensuring that all key-value pairs for a given key are sent to the same Reducer. This is crucial for correct aggregation and computation during the Reducer phase.

Reducer:

Grouping and Aggregation: The Reducer takes the intermediate key-value pairs produced by the Mappers, groups them by key (based on the output of the Partitioner), and then applies a user-defined Reduce function to each group. The result is typically written to an output file.

Shuffle and Sort: After the Mapper phase, there's a shuffle and sort step where the MapReduce framework sorts and groups the intermediate key-value pairs before sending them to the Reducers. This ensures that a Reducer processes together all values associated with the same key.

These mathematical algorithms may include the following –

- Sorting
- Searching
- Indexing
- TF-IDF

Sorting:

Sorting is one of the basic MapReduce algorithms to process and analyze data. MapReduce implements sorting algorithm to automatically sort the output key-value pairs from the mapper by their keys.

- Sorting methods are implemented in the mapper class itself.
- In the Shuffle and Sort phase, after tokenizing the values in the mapper class, the Context class (user-defined class) collects the matching valued keys as a collection.
- To collect similar key-value pairs (intermediate keys), the Mapper class takes the help of RawComparator class to sort the key-value pairs.

- The set of intermediate key-value pairs for a given Reducer is automatically sorted by Hadoop to form key-values (K2, {V2, V2, ...}) before they are presented to the Reducer.

Searching:

Searching plays an important role in MapReduce algorithm. It helps in the combiner phase (optional) and in the Reducer phase. Let us try to understand how Searching works with the help of an example.

Compression :

Data Compression: To optimize data transfer and storage, MapReduce frameworks often support data compression. Intermediate data and output data can be compressed to reduce disk I/O and network bandwidth usage.

In summary, MapReduce is a powerful framework for processing large-scale data by breaking it into smaller tasks (Mappers), aggregating results (Reducers), and allowing for data manipulation and searching through user-defined logic. The additional components like Combiners, Partitioners, Sorting, and Compression are used to optimize the process and improve performance in distributed computing environments

Example

The following example shows how MapReduce employs Searching algorithm to find out the details of the employee who draws the highest salary in a given employee dataset.

- Let us assume we have employee data in four different files – A, B, C, and D. Let us also assume there are duplicate employee records in all four files because of importing the employee data from all database tables repeatedly. See the following illustration.

name, salary	name, salary	name, salary	name, salary
satish, 26000	gopal, 50000	satish, 26000	satish, 26000
Krishna, 25000	Krishna, 25000	kiran, 45000	Krishna, 25000
Satishk, 15000	Satishk, 15000	Satishk, 15000	manisha, 45000
Raju, 10000	Raju, 10000	Raju, 10000	Raju, 10000

- The Map phase processes each input file and provides the employee data in key-value pairs (<k, v> : <emp name, salary>). See the following illustration.

<satish, 26000>	<gopal, 50000>	<satish, 26000>	<satish, 26000>
<Krishna, 25000>	<Krishna, 25000>	<kiran, 45000>	<Krishna, 25000>
<Satishk, 15000>	<Satishk, 15000>	<Satishk, 15000>	<manisha, 45000>
<Raju, 10000>	<Raju, 10000>	<Raju, 10000>	<Raju, 10000>

- **The combiner phase** (searching technique) will accept the input from the Map phase as a key-value pair with employee name and salary. Using searching technique, the combiner will check all the employee salary to find the highest salaried employee in each file. See the following snippet.

```

•
• <k: employee name, v: salary>
• Max= the salary of an first employee. Treated as
  max salary
•
• if(v(second employee).salary > Max){
•   Max = v(salary);
• }
•
• else{
•   Continue checking;
• }

```

The expected result is as follows –

- **Reducer phase** – Form each file, you will find the highest salaried employee. To avoid redundancy, check all the <k, v> pairs and eliminate duplicate entries, if any. The same algorithm is used in between the four <k, v> pairs, which are coming from four input files. The final output should be as follows –

```

• <gopal, 50000>

```

Indexing

Normally indexing is used to point to a particular data and its address. It performs batch indexing on the input files for a particular Mapper. The indexing technique that is normally used in MapReduce is known as inverted index. Search engines like Google and Bing use inverted indexing technique. Let us try to understand how Indexing works with the help of a simple example.

Chapter 4

INTRODUCTION TO HIVE

Hive is an open-source data warehousing and SQL-like query language system that facilitates the management and querying of large datasets stored in distributed storage, primarily Hadoop Distributed File System (HDFS). It was developed by Facebook and later contributed to the Apache Software Foundation, becoming part of the Apache Hadoop ecosystem. Here's an introduction to Hive:

Purpose and Background:

Hive was created to provide a familiar SQL-like interface for working with big data stored in Hadoop. It was designed for users who are already familiar with SQL, making it easier for them to work with and analyze large-scale datasets without having to learn new programming languages or paradigms.

Data Warehousing:

Hive is often used for data warehousing tasks, where it allows users to structure and query data stored in HDFS. It provides a way to define schemas and tables on top of raw data files, making them more accessible for querying and analysis.

HiveQL (HQL):

Hive provides a high-level query language called HiveQL or HQL, which is similar to SQL. Users can write SQL-like queries to retrieve, transform, and analyze data. Hive translates these queries into MapReduce jobs that can be executed on the Hadoop cluster.

Metastore:

Hive has a metastore that stores metadata about tables, partitions, columns, and data types. This metadata enables Hive to manage the structured data on top of the unstructured data stored in HDFS. It also allows for schema evolution, where you can modify table structures without changing the data files.

User-Defined Functions (UDFs)

Hive supports the use of User-Defined Functions (UDFs) written in Java, Python, or other languages. UDFs enable users to extend Hive's functionality by implementing custom functions for data transformation and analysis.

Extensibility:

Hive is extensible and can be integrated with other Hadoop ecosystem components and tools like HBase, Spark, and Pig. This allows users to combine the strengths of various technologies to meet their specific data processing and analysis requirements.

Optimizations :

Over time, Hive has introduced various performance optimizations, such as query optimization, query caching, and vectorization, to improve query execution times and efficiency.

Use Cases:

Hive is commonly used for batch processing and data warehousing tasks in scenarios where data is too large to fit into traditional relational databases. It is well-suited for tasks like log analysis, data aggregation, ETL (Extract, Transform, Load) processes, and business intelligence reporting on big data.

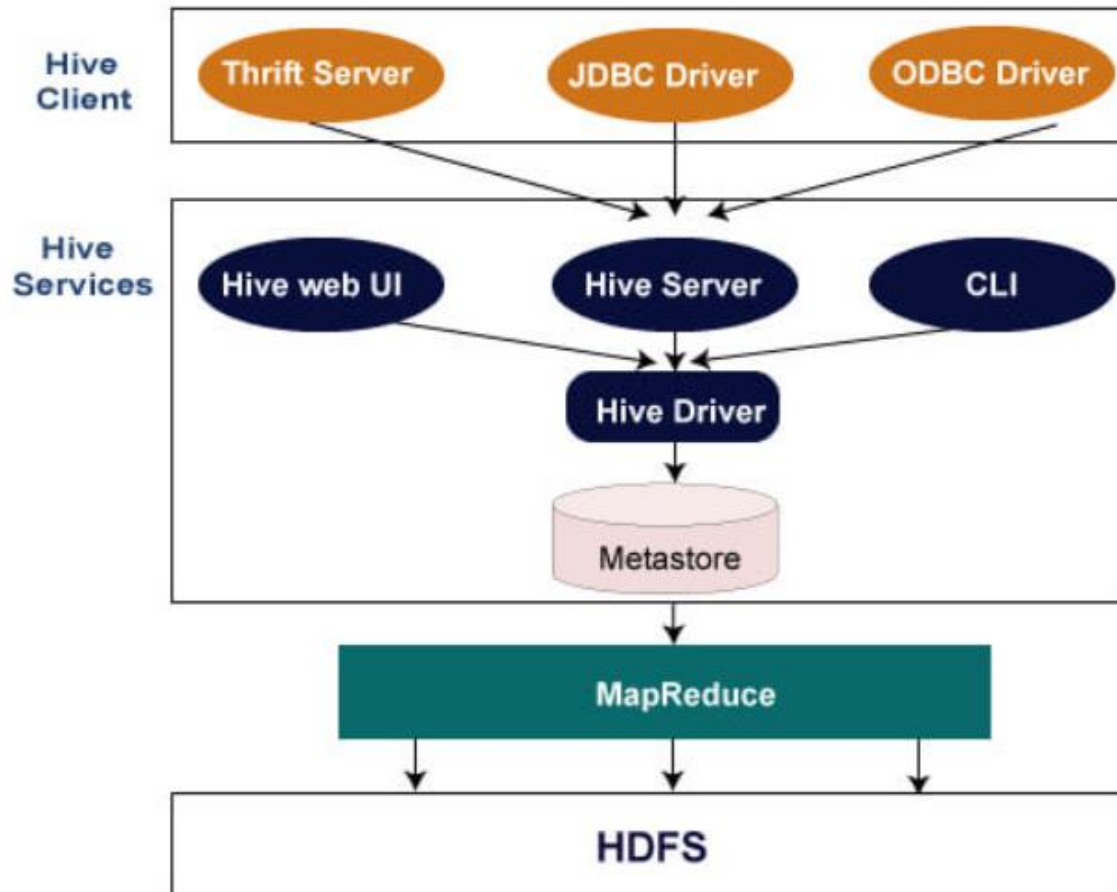
Hive Ecosystem:

Hive is part of the broader Hadoop ecosystem and can be integrated with other Hadoop components like HBase for NoSQL capabilities, Spark for real-time and iterative processing, and Pig for data transformation.

In summary, Hive is a powerful tool for working with big data stored in Hadoop clusters. It provides a SQL-like interface, making it accessible to users with SQL skills, and it is widely used for data warehousing, batch processing, and analytics tasks on large datasets in distributed environments. Its extensibility and integration with other Hadoop components make it a valuable tool in the world of big data processing.

Hive Architecture

The following architecture explains the flow of submission of query into Hive.



Hive Client:

Hive allows writing applications in various languages, including Java, Python, and C++. It supports different types of clients, such as:-

- **Thrift Server** - It is a cross-language service provider platform that serves the request from all those programming languages that supports Thrift.
- **JDBC Driver** - It is used to establish a connection between hive and Java applications. The JDBC Driver is present in the class `org.apache.hadoop. Hive.jdbc.HiveDriver`.
- **ODBC Driver** - It allows the applications that support the ODBC protocol to connect to Hive.

Hive Services:

The following are the services provided by Hive:-

- **Hive CLI** - The Hive CLI (Command Line Interface) is a shell where we can execute Hive queries and commands.
- **Hive Web User Interface** - The Hive Web UI is just an alternative of Hive CLI. It provides a web-based GUI for executing Hive queries and commands.
- **Hive MetaStore** - It is a central repository that stores all the structure information of various tables and partitions in the warehouse. It also includes metadata of column and its type information, the serializers and deserializers which is used to read and write data and the corresponding HDFS files where the data is stored.
- **Hive Server** - It is referred to as Apache Thrift Server. It accepts the request from different clients and provides it to Hive Driver.
- **Hive Driver** - It receives queries from different sources like web UI, CLI, Thrift, and JDBC/ODBC driver. It transfers the queries to the compiler.
- **Hive Compiler** - The purpose of the compiler is to parse the query and perform semantic analysis on the different query blocks and expressions. It converts HiveQL statements into MapReduce jobs.
- **Hive Execution Engine** - Optimizer generates the logical plan in the form of DAG of map-reduce tasks and HDFS tasks. In the end, the execution engine executes the incoming tasks in the order of their dependencies

Hive's architecture is designed to provide a SQL-like interface and a high-level abstraction for working with large datasets stored in Hadoop clusters. It uses a combination of metadata management, query translation, execution engines, and user-defined functions to enable users to process, analyze, and query big data effectively in a distributed environment.

File formats

Hive supports various file formats for storing and managing data within its tables. The choice of file format can significantly impact performance, storage efficiency, and compatibility with other data processing tools. Here are some of the commonly used file formats in Hive.

TextFile:

The TextFile format stores data in plain text files, where each row is typically delimited by a newline character, and columns are separated by a delimiter such as a tab or comma. While

TextFile is human-readable, it may not be the most efficient format for large-scale data processing due to its lack of compression and limited support for complex data types.

SequenceFile:

SequenceFile is a binary file format native to Hadoop. It's a compact, efficient format that supports both key-value pairs and complex data structures like arrays and maps. SequenceFiles can be compressed, making them suitable for storing large datasets.

ORC (Optimized Row Columnar):

ORC is a columnar storage file format specifically optimized for Hive. It offers excellent compression and performance improvements over TextFile and SequenceFile. ORC files store data in a columnar format, making it easier for Hive to skip irrelevant data when executing queries. ORC is the default file format for Hive because of its speed and efficiency.

Parquet:

Parquet is another columnar storage format that is highly optimized for analytics workloads. It provides excellent compression, schema evolution support, and compatibility with other data processing tools like Apache Spark and Apache Impala. Hive can read and write Parquet files.

Avro:

Avro is a row-based data serialization format that provides a compact binary representation. Avro files contain a schema within them, which makes them self-describing and schema evolution-friendly. Hive can read and write Avro files, making it suitable for certain use cases where schema flexibility is essential.

RCFile (Record Columnar File):

RCFile is another columnar file format designed for Hive. It stores data in a columnar manner, similar to ORC and Parquet, which allows for efficient compression and predicate pushdown optimizations. While not as commonly used as ORC and Parquet, RCFile is still an option for specific scenarios.

JSON and JSON SerDe

Hive can work with JSON data using the built-in JSON SerDe (Serializer/Deserializer). While JSON is not a file format per se, the JSON SerDe allows Hive to parse and process JSON data stored in TextFile or other formats. It's useful when dealing with semi-structured data.

HBase:

Hive can also read data from Apache HBase, a NoSQL database, as if it were a table. HBase is typically used when low-latency access to data is required.

The choice of file format in Hive should take into consideration factors such as data volume, query performance, schema evolution requirements, and compatibility with other tools in your data ecosystem. ORC and Parquet are often the preferred choices for analytical workloads due to their columnar storage and compression benefits, while TextFile and SequenceFile are simpler and more human-readable but may be less efficient for large-scale data processing.

Hive Query Language Statements

Hive Query Language (HiveQL) is a SQL-like query language used for querying and managing data in Apache Hive, a data warehousing and SQL-like query system built on top of Hadoop. HiveQL allows you to perform various operations on structured and semi-structured data stored in Hive tables. Here are some common HiveQL statements and their descriptions

Creating a Database:

```
CREATE DATABASE database_name;
```

Creates a new database in Hive.

Switching to a Database:

Switches the current database context to the specified database.

```
USE database_name;
```

Creating a Table:

Creates a new table in Hive with the specified columns and data types

```
CREATE TABLE table_name (  
    column1 datatype,  
    column2 datatype,  
    ...  
);
```

Loading Data into a Table:

Loads data from a local file (on the Hive server) into a Hive table.

LOAD DATA LOCAL INPATH 'input_file' INTO TABLE table_name;

Inserting Data into a Table:

Inserts data into a Hive table row by row.

INSERT INTO TABLE table_name VALUES (value1, value2, ...);

Selecting Data

Retrieves data from a table, optionally filtering rows based on a condition.

*SELECT * FROM table_name WHERE condition;*

Aggregating Data:

Performs aggregation functions like COUNT, SUM, AVG, etc., on data grouped by one or more columns

SELECT COUNT(), SUM(column) FROM table_name GROUP BY grouping_column;*

Joining Tables:

Combines data from multiple tables based on a specified column(s).

*SELECT * FROM table1 JOIN table2 ON table1.column = table2.column;*

Sorting Data:

Sorts the result set in ascending (ASC) or descending (DESC) order.

*SELECT * FROM table_name ORDER BY column_name [ASC|DESC];*

Filtering Data:

Filters data based on a specified condition using operators like =, <, >, etc.

*SELECT * FROM table_name WHERE column_name operator value;*

Creating Views:

Creates a virtual table (view) based on the result of a SELECT query.

*CREATE VIEW view_name AS SELECT * FROM table_name WHERE condition;*

Altering Table Structure:

Adds a new column to an existing table

```
ALTER TABLE table_name ADD COLUMN new_column datatype;
```

Dropping Tables and Databases:

Deletes a table or an entire database along with its tables.

```
DROP TABLE table_name;  
DROP DATABASE database_name;
```

Describing Tables:

Retrieves metadata about a table, including its column names and data types.

```
DESCRIBE table_name;
```

Managing Partitions:

Adds a new partition to a partitioned table.

```
ALTER TABLE table_name ADD PARTITION (partition_column=value);
```

Partitions

In Hive, partitions are a way to organize and optimize data storage and retrieval in tables, especially for large datasets. They allow you to divide your table into smaller, more manageable parts based on one or more columns, typically categorical or discrete columns, such as date, country, or category. Here are some key aspects of partitions in Hive

Partitioning Column(s) :

Partitioning is based on one or more columns in the table. These columns are known as partition keys. For example, if you have a sales data table, you might partition it by the "year" and "month" columns.

Partitioning Types:

Hive supports two types of partitioning:

Static Partitioning: You explicitly define and load data into each partition. It's suitable when you have control over how data is organized.

Dynamic Partitioning: Hive automatically organizes data into partitions based on the values of the partitioning columns during data insertion. Dynamic partitioning is useful for scenarios where data is continuously ingested.

Partitioned Tables:

To create a partitioned table, you specify the partition keys when defining the table schema. For example:

```
CREATE TABLE sales (  
    product_id INT,  
    amount DOUBLE  
)  
PARTITIONED BY (year INT, month INT);
```

Loading Data into Partitions:

When loading data into a partitioned table, you need to specify the partition keys in your LOAD or INSERT statement. For example:

```
Static partitioning  
INSERT INTO TABLE sales PARTITION (year=2023, month=9) VALUES (1, 100.0);
```

```
-- Dynamic partitioning  
INSERT INTO TABLE sales VALUES (1, 200.0, 2023, 10);
```

Benefits of Partitions:

Efficient Querying: Partitioning helps Hive optimize query performance by allowing it to skip irrelevant partitions when executing queries. This reduces the amount of data scanned.

Data Organization: Data is organized logically, making it easier to manage and query, especially for large datasets.

Parallel Processing: Partitioned tables enable parallel processing of partitions, leading to faster query execution.

Schema Evolution: Partitioning can help with schema evolution. You can add or remove partitions without affecting the entire dataset.

Partitioning Best Practices:

- Choose partition keys that make sense for your use case and query patterns
- Avoid creating too many partitions, as it can lead to increased overhead
- Use a well-thought-out directory structure in HDFS for partitioned data. Hive uses directories to physically store partitioned data.

Partitioning and Data Types:

- Hive supports partitioning by a wide range of data types, including numeric, string, date, and timestamp types
- Be mindful of the data type you use for partition keys, as it can affect sorting and querying behavior

Partitions in Hive are a valuable feature for organizing and optimizing data storage and retrieval in large tables. They help improve query performance, data management, and schema evolution. Whether you use static or dynamic partitioning depends on your specific use case and data loading requirements.

Bucketing

Bucketing in Hive is a technique for optimizing data storage and query performance by dividing a table into smaller, more manageable parts called buckets. Bucketing is particularly useful when dealing with large datasets and is often used in conjunction with partitioning. Here are the key aspects of bucketing in Hive:

Bucketing vs. Partitioning

While both bucketing and partitioning aim to improve query performance, they serve different purposes:

Partitioning: Divides a table into separate subdirectories or partitions based on one or more columns, typically categorical or discrete columns (e.g., date, country)

Bucketing: Divides a table into smaller, roughly equal-sized parts based on a hash function applied to one or more columns, typically numeric or continuous columns (e.g., IDs, timestamps).

Bucketing Columns:

In Hive, you specify the bucketing columns when creating a table. These columns are used to calculate a hash value that determines the bucket to which each row belongs.

Bucketing Syntax:

To create a bucketed table, you include the `CLUSTERED BY` clause along with the number of buckets in the `CREATE TABLE` statement. For example:

```
CREATE TABLE sales_bucketed (  
    product_id INT,  
    amount DOUBLE  
)  
CLUSTERED BY (product_id) INTO 10 BUCKETS.
```

Bucketing Process:

When data is inserted into a bucketed table, Hive calculates a hash value based on the specified bucketing columns and assigns each row to a bucket. Rows with the same hash value are placed in the same bucket.

Query Optimization:

Bucketing can significantly improve query performance, especially when used in conjunction with partitioning. When querying a bucketed table, Hive knows which bucket to scan based on the hash value, reducing the amount of data scanned and improving query execution times.

Bucketing Best Practices:

- Choose bucketing columns that evenly distribute data to buckets. Good candidates are columns with high cardinality (many unique values)
- Avoid using columns with low cardinality for bucketing, as it may lead to skewed data distribution across buckets
- The number of buckets should ideally be a multiple of the number of reducers you plan to use for queries to maximize parallelism.

Bucketing and Partitioning:

You can use both bucketing and partitioning together to further optimize your tables. For example, you can partition a sales table by year and month and then bucket it by product_id.

Bucketing and Sorting:

Hive supports bucketed tables with sort orders on one or more columns. Sorting the data within each bucket can further improve query performance when querying based on sorted columns.

In summary, bucketing in Hive is a technique for dividing tables into smaller buckets based on hash values calculated from specified columns. It's a valuable optimization technique for improving query performance, especially in scenarios involving large datasets. Properly chosen bucketing columns and a suitable number of buckets are crucial for effective bucketing and query optimization.

Views

In Hive, views are virtual tables that provide a way to query and access data stored in one or more underlying base tables without duplicating or moving the data. Views are especially useful for simplifying complex queries, enforcing security policies, and providing a logical data model to users. Here's how views work in Hive.

Creating a View

To create a view in Hive, you use the `CREATE VIEW` statement, which includes the view name and the query that defines the view. The query specifies which columns and rows from the underlying tables should be included in the view, for example.

```
CREATE VIEW my_view AS
SELECT column1, column2
FROM my_table
WHERE column3 = 'some_value';
```

View Definition:

The view definition is a SQL query that specifies the transformation or subset of data from one or more base tables. This query can include filters, joins, aggregations, and any other SQL operations.

Querying a View:

Once a view is created, you can query it like any other table using `SELECT` statements. Hive will execute the underlying query and return the results as if they were from a physical table.

Updating Views:

In Hive, views are read-only by default. This means you cannot use a view to modify the data in the underlying tables. However, you can create an "updatable view" by adding an `WITH CHECK OPTION` clause to the view definition, which allows you to insert, update, or delete data through the view. Be cautious when using updatable views, as they can impact data integrity.

Dropping Views:

You can drop a view using the `DROP VIEW` statement. This removes the view definition but does not affect the underlying data in the base tables. For example:

```
DROP VIEW my_view;
```

View Security and Permissions:

Hive views respect the underlying table's permissions and security settings. Users must have appropriate permissions on the view and the underlying tables to access the data. Additionally, Hive supports column-level security with views, allowing you to control which columns users can access.

View Dependencies:

Hive keeps track of dependencies between views and underlying tables. If a view is dependent on another view or table, it must be recreated if the underlying objects change.

In summary, Hive views provide a convenient way to simplify complex queries, enforce security, and present a logical data model to users. They are a valuable tool for working with Hive tables and abstracting the complexities of underlying data structures.

Subqueries / Nested queries

Nested Queries:

- A nested query in SQL contains a query inside another query. The outer query will use the result of the inner query. For instance, a nested query can have two SELECT statements, one on the inner query and the other on the outer query.

```
SELECT ID, NAME FROM EMPLOYEES
WHERE ID IN (SELECT EMPLOYEE_ID FROM AWARDS)
```

A horizontal curly bracket is positioned below the subquery '(SELECT EMPLOYEE_ID FROM AWARDS)'. Below the bracket, the text 'Nested Query' is centered.

Nested Query

Types of Nested Queries in SQL :

Subqueries, also known as nested queries, are queries that are embedded within another query in Hive. Subqueries allow you to perform complex operations by breaking them down into smaller, more manageable parts. Hive supports subqueries in various contexts, such as in the SELECT, FROM, WHERE, and HAVING clauses of a query. Here are some examples of how to use subqueries in Hive

Nested queries in SQL can be classified into two different types:

- Unco-related Nested Queries (Independent Nested Queries)
- Co-related Nested Queries

Independent Nested Queries/ Unco-related Nested Queries:

In independent nested queries, the execution order is from the innermost query to the outer query. An outer query won't be executed until its inner query completes its execution. The outer query uses the result of the inner query. Operators such as **IN**, **NOT IN**, **ALL**, and **ANY** are used to write independent nested queries.

- If a subquery does use any references from outer query then it would be called as Independent Subquery.
- The IN operator checks if a column value in the outer query's result is **present** in the inner query's result. The final result will have rows that satisfy the IN condition.
- The NOT IN operator checks if a column value in the outer query's result is **not present** in the inner query's result. The final result will have rows that satisfy the NOT IN condition.
- The ALL operator compares a value of the outer query's result with **all the values** of the inner query's result and returns the row if it matches all the values.
- The ANY operator compares a value of the outer query's result with all the inner query's result values and returns the row if there is a match with any value.



Co-related Nested Queries

In co-related nested queries, the inner query uses the values from the outer query to execute the inner query for every row processed by the outer query. The co-related nested queries run slowly because the inner query is executed for every row of the outer query's result.

- Outer query needs to be executed before the inner query.
- The inner query is executed separately for each row of the outer query.
- here are different types of set comparison operators like EXISTS, IN and UNIQUE. SQL also supports op ANY and op ALL, where op means arithmetic comparison operators such as <, <=, =, <>, >=, >. SOME are also one of the set comparison operators but it is similar to ANY.

- A correlated subquery is evaluated once for each row processed by the parent statement. The parent statement can be a **SELECT**, **UPDATE**, or **DELETE** statement.

Syntax:

```
SELECT column1, column2, ....  
FROM table1 outer  
WHERE column1 operator  
(SELECT column1, column2  
FROM table2  
WHERE expr1 = outer.expr2);
```

Example

We will use the Employees and Awards table below to understand independent and co-related nested queries.

id	name	salary	role
1	Augustine Hammond	10000	Developer
2	Perice Mundford	10000	Manager
3	Cassy Delafoy	30000	Developer
4	Garwood Saffen	40000	Manager
5	Faydra Beaves	50000	Developer

id	employee_id	award_date
1	1	2022-04-01
2	3	2022-05-01

Example 1: IN

- Select all employees who won an award

Syntax :

```
SELECT id, name FROM employees  
WHERE id IN (SELECT employee_id FROM awards);
```

Output:

Output

id	name
2	Perice Mundford
4	Garwood Saffen
5	Faydra Beaves

Example 2: NOT IN

Select all employees who never won an award.

```
SELECT id, name FROM employees  
WHERE id NOT IN (SELECT employee_id) FROM awards);
```

Output :

id	name
1	Augustine Hammond
3	Cassy Delafoy

Example 3: ALL

- Select all **Developers** who earn more than all the **Managers**

```
SELECT * FROM employees  
WHERE role = 'Developer'  
AND salary > ALL (
```

SELECT salary FROM employees WHERE role = 'Manager'

);

Output :

id	name	salary	role
5	Faydra Beaves	50000	Developer

Example 4: ANY

- Select all **Developers** who earn more than any **Manager**

*SELECT * FROM employees*

WHERE role = 'Developer'

AND salary > ANY (

SELECT salary FROM employees WHERE role = 'Manager'

);

Output :

id	name	salary	role
5	Faydra Beaves	50000	Developer
3	Cassy Delafoy	30000	Developer

Co-related Nested Queries

- Select all employees whose salary is above the average salary of employees in their role.

*SELECT * FROM employees emp1*

WHERE salary > (

SELECT AVG(salary)

FROM employees emp2

WHERE emp1.role = emp2.role

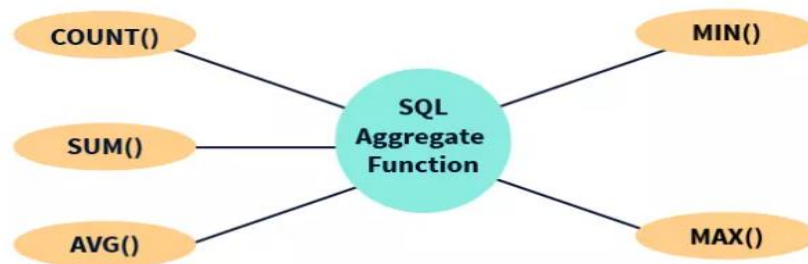
);

Output :

id	name	salary	role
4	Garwood Saffen	40000	Manager
5	Faydra Beaves	50000	Developer

Aggregate Functions in SQL

An aggregate function in SQL performs a calculation on multiple values and returns a single value. SQL provides many aggregate functions that include avg, count, sum, min, max, etc. An aggregate function ignores NULL values when it performs the calculation, except for the count function.



1. COUNT() Function

The COUNT() aggregate function returns the total number of rows from a database table that matches the defined criteria in the SQL query.

Syntax

COUNT(*) OR COUNT(COLUMN_NAME)

Example :

The given table named EMP_DATA consists of data concerning 10 employees working in the same organization in different departments.

Employee_ID	Name	Department	Salary
1	Ram	Marketing	80000
2	Henry	Production	76000
3	Disha	R&D	76000
4	Helen	R&D	84000
5	Meera	Marketing	80000
6	Ashish	Production	64000
7	Bob	Production	60000
8	Hari	R&D	60000
9	Preeti	Marketing	NULL
10	Mark	Production	66000

1. Suppose you want to know the total number of employees working in the organization. You can do so by the below-given query.

SELECT COUNT() FROM EMP_DATA;*

As COUNT(*) returns the total number of rows and the table named EMP_DATA provided above consists of 10 rows, so the COUNT(*) function returns 10. The output is printed as shown below.

Output: 10

Note: Except for COUNT(*), all other SQL aggregate functions ignore NULL values.

2. Suppose you need to count the number of people who are getting a salary. The query given below can help you achieve this.

SELECT COUNT(Salary) FROM EMP_DATA;

Output : 9

Here, the Salary column is passed as a parameter to the COUNT() function, and hence, this query returns the number of non NULL values from the column Salary, i.e. 9.

3. Suppose you need to count the number of distinct departments present in the organization. The following query can help you achieve this.

```
SELECT COUNT(DISTINCT Department) FROM EMP_DATA;
```

Output: 3

The above query returns the total number of distinct non NULL values over the column Department i.e. 3 (Marketing, Production, R&D). The DISTINCT keyword makes sure that only non-repetitive values are counted.

4. What if you want to calculate the number of people whose salaries are more than a given amount(say 70,000)? Check out the example below.

```
SELECT COUNT(Salary) WHERE Salary >= 70000 FROM EMP_DATA;
```

Output : 5

The query returns the number of rows where the salary of the employee is greater than or equal to 70,000 i.e 5.

2. SUM() Function

The SUM() function takes the name of the column as an argument and returns the sum of all the non NULL values in that column. It works only on numeric fields(i.e the columns contain only numeric values). When applied to columns containing both non-numeric(ex - strings) and numeric values, only numeric values are considered. If no numeric values are present, the function returns 0.

Syntax:

The function name is SUM() and the name of the column to be considered is passed as an argument to the function.

```
SUM(COLUMN_NAME)
```

Example:

1. Suppose you need to build a budget for the organization and you need to know the total amount needed to provide salaries to all the employees. To calculate the sum of all the values present in column Salary. You can refer to the below-given example.

SELECT SUM(Salary) FROM EMP_DATA;

Output :646000

The above-mentioned query returns the sum of all non-NULL values over the column Salary i.e $80000 + 76000 + 76000 + 84000 + 80000 + 64000 + 60000 + 60000 + 66000 = 646000$

2. What if you need to consider only distinct salaries? The following query will help you achieve that.

SELECT SUM(DISTINCT Salary) FROM EMP_DATA;

Output : 430000

The DISTINCT keyword makes sure that only non-repetitive values are considered. The query returns the sum of all distinct non NULL values over the column Salary i.e. $80000 + 76000 + 84000 + 64000 + 60000 + 66000 = 430000$.

3. Suppose you need to know the collective salaries for each department(say Marketing). The query given below can help you achieve this.

SELECT SUM(SALARY) FROM EMP_DATA WHERE Department = "Marketing";

Output :160000

The query returns the sum of salaries of employees who are working in the Marketing Department i.e $80000 + 80000 = 160000$.

Note: There are 3 rows consisting of Marketing as Department value but the third value is a NULL value. Thus, the sum is returned considering only the first two entries having Marketing as Department.

3.AVG() Function

The AVG() aggregate function uses the name of the column as an argument and returns the average of all the non NULL values in that column. It works only on numeric fields(i.e the columns contain only numeric values).

Note: When applied to columns containing both non-numeric (ex - strings) and numeric values, only numeric values are considered. If no numeric values are present, the function returns 0.

Syntax:

The function name is AVG() and the name of the column to be considered is passed as an argument to the function.

AVG(COLUMN_NAME)

Example:

1. To obtain the average salary of an employee of an organization, the following query can be used.

SELECT AVG(Salary) FROM EMP_DATA;

Output : 71777.77777

Here, the column name Salary is passed as an argument and thus the values present in column Salary are considered. The above query returns the average of all non NULL values present in the Salary column of the table.

Average = $(80000 + 76000 + 76000 + 84000 + 80000 + 64000 + 60000 + 60000 + 66000) / 9$
 $= 646000 / 9 = 71777.77777$

2. If you need to consider only distinct salaries, the following query will help you out.

SELECT AVG(DISTINCT Salary) FROM EMP_DATA;

Output : 71666.66666

The query returns the average of all non NULL distinct values present in the Salary column of the table.

Average = $(80000 + 76000 + 84000 + 64000 + 60000 + 66000) / 6 = 430000 / 6 = 71666.66666$.

4. MIN() Function

The MIN() function takes the name of the column as an argument and returns the minimum value present in the column. MIN() returns NULL when no row is selected.

Syntax:

The function name is MIN() and the name of the column to be considered is passed as an argument to the function.

MIN(COLUMN_NAME)

Example;

1. Suppose you want to find out what is the minimum salary that is provided by the organization. The MIN() function can be used here with the column name as an argument.

SELECT MIN(Salary) FROM EMP_DATA;

Output :60000

The query returns the minimum value of all the values present in the mentioned column i.e 60000.

2. Suppose you need to know the minimum salary of an employee belonging to the Production department. The following query will help you achieve that.

SELECT MIN(Salary) FROM EMP_DATA WHERE Department = "Production";

Output:60000

The query returns the minimum value of all the values present in the mentioned column and has Production as Department value i.e 60000.

5. MAX() Function

The MAX() function takes the name of the column as an argument and returns the maximum value present in the column. MAX() returns NULL when no row is selected.

Syntax:

The function name is MAX() and the name of the column to be considered is passed as an argument to the function.

MAX(COLUMN_NAME)

Example :

1. Suppose you want to find out what is the maximum salary that is provided by the organization. The MAX() function can be used here with the column name as an argument.

SELECT MAX(Salary) FROM EMP_DATA;

Output : 84000

The query returns the maximum value of all the values present in the mentioned column i.e 84000.

2. Suppose you need to know the maximum salary of an employee belonging to the R&D department. The following query will help you achieve that.

SELECT MAX(Salary) FROM EMP_DATA WHERE Department="R&D";

Output : 84000

The query returns the maximum value of all the values present in the mentioned column and has R&D as Department value i.e 84000.

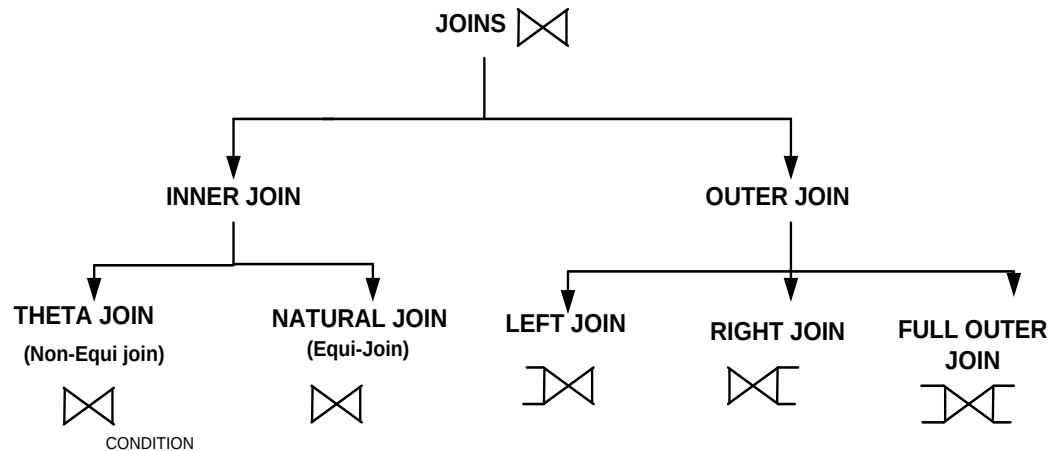
Join operations

A SQL Join statement combines data or rows from two or more tables based on a common field between them.

- SQL Joins are mostly used when a user is trying to extricate data from multiple tables (which have one-to-many or many-to-many relationships with each other) at one time.
- Large databases are often prone to data redundancy, i.e., the creation of repetitive data anomalies by insertion, deletion, and updation. But by using SQL Joins, we promote database normalization, which reduces data redundancy and eliminates redundant data.
- In SQL joins are mainly two types they are INNER JOINS and OUTER JOINS. Join can be represented by using below symbol.



In SQL, joins can be classified as either **equi-joins** or **non-equi-joins** based on the conditions used to match rows between tables.



Equi-Join: An equi-join is a type of join where the matching condition is based on equality between columns from different tables. It matches rows where the specified columns have the same values.

- the Equi Join in SQL returns only the data in all the tables we are comparing based on the common column field. It does not display null or unmatchable data.
- The equality operator in the Equi Join operation is used to refer to the equality in the WHERE clause. However, it returns the same result when we use the JOIN keyword with the ON clause along with column names and their respective tables

Example :

*SELECT **

FROM TableName1, TableName2

WHERE TableName1.ColumnName = TableName2.ColumnName;

OR

*SELECT **

FROM TableName1

JOIN TableName2

ON TableName1.ColumnName = TableName2.ColumnName;

- An **equi join** is any JOIN operation that uses only and only the equals sign. If there is a query with more than one join condition, out of which one condition has an equals sign, and the other doesn't, then this query would be considered a non-equi join in SQL.

Non-equi join :

- A non-equi join, also known as a range join or a theta join, is a type of join operation where the joining condition involves operators other than equality, such as greater than (>), less than (<), greater than or equal to (>=), less than or equal to (<=), or not equal to (!= or <>).
- Non-Equi Join is also a type of INNER Join in which we need to retrieve data from multiple tables. In a non-equi join, rows are matched based on a range of values rather than a direct equality. Non-equi joins are less common and are often used to solve specific data analysis problems.

However, we use the Non-Equi joins for the below-mentioned reasons-

- Retrieving data matching in a range of values.
- Checking for duplicate data between tables.
- For calculating totals.

Here's an example

*SELECT **

FROM TableName1, TableName2

WHERE TableName1.columnName [> | < | >= | <= | != | BETWEEN] table_name2.column;

INNER JOIN in SQL

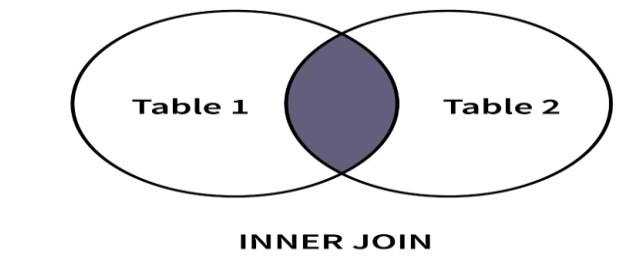
An inner join returns only the rows that have matching values in both tables being joined. It combines rows from two tables where the join condition is satisfied. The syntax for an inner join is as follows:

*SELECT **

FROM table1

INNER JOIN table2

ON table1.column = table2.column;



NATURAL JOIN

- SQL Natural Join is a type of Inner join based on the condition that columns having the same name and datatype are present in both the tables to be joined.

```
SELECT *  
  
FROM table-1  
  
NATURAL JOIN table-2;
```

OUTER JOINS in SQL

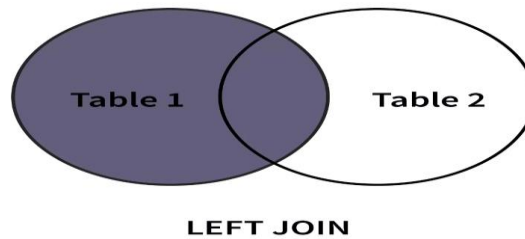
SQL Outer joins give both matched and unmatched rows of data depending on the type of outer joins. These types of outer joins are sub-divided into the following types:

- **Left Outer Join**
- **Right Outer Join**
- **Full Outer Join**

Left Join (or Left Outer Join):

- A left join returns all the rows from the left table and the matching rows from the right table. If there are no matching rows in the right table, NULL values are included for the columns of the right table. The syntax for a left join is as follows:

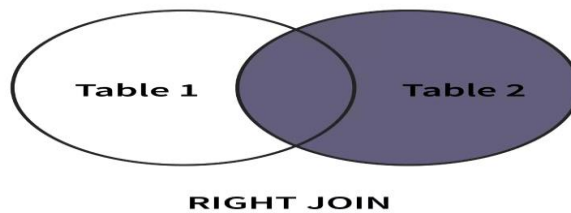
```
SELECT *  
  
FROM table1  
  
LEFT JOIN table2  
  
ON table1.column = table2.column;
```



Right Join (or Right Outer Join):

A right join returns all the rows from the right table and the matching rows from the left table. If there are no matching rows in the left table, NULL values are included for the columns of the left table. The syntax for a right join is as follows:

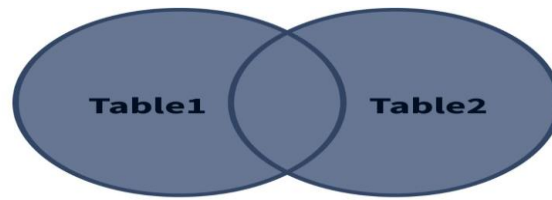
```
SELECT *  
  
FROM table1  
  
RIGHT JOIN table2  
  
ON table1.column = table2.column;
```



Full Join (or Full Outer Join):

A full join returns all the rows from both the left and right tables. If there are no matching rows in either table, NULL values are included for the columns of the non-matching table.

```
SELECT *  
  
FROM table1  
  
FULL JOIN table2  
  
ON table1.column = table2.column;
```



FULL JOIN

Theta join :

In SQL, a theta join, also known as a non-equi join or a range join, is a type of join operation where the joining condition involves comparison operators other than equality (=).

The syntax for a theta join typically involves using the JOIN keyword followed by the tables being joined and the join condition with the desired comparison operator(s). Here's an example:

```
SELECT *
FROM table1
JOIN table2
ON table1.column1 < table2.column2;
```

Examples :

Customer

<i>Id</i>	<i>Cust_Name</i>	<i>Address</i>
<i>1</i>	<i>Ram</i>	<i>A</i>
<i>2</i>	<i>Raj</i>	<i>B</i>
<i>3</i>	<i>Rani</i>	<i>C</i>

Orders

<i>Or_Id</i>	<i>amount</i>	<i>Cust_id</i>
<i>601</i>	<i>1000</i>	<i>1</i>
<i>602</i>	<i>2000</i>	<i>4</i>
<i>603</i>	<i>3000</i>	<i>2</i>

1. Inner join :

```
SELECT Orders.Or_Id ,Customers.Cust_Name  
FROM Orders  
INNER JOIN Customers  
ON Orders.Cust_Id=Customers.Id;
```

Output

Or_Id	Cust_Name
601	Ram
603	Raj

LEFT JOIN

```
SELECT Orders.Or_Id ,Customers.Cust_Name  
FROM Orders  
LEFT JOIN Customers  
ON Orders.Cust_Id=Customers.Id;
```

Out put:

Or_Id	Cust_Name
601	Ram
602	NULL
603	Raj

Right Join

```
SELECT Orders.Or_Id ,Customers.Cust_Name  
FROM Orders  
RIGHT JOIN Customers  
ON Orders.Cust_Id=Customers.Id;
```

Out put:

Or_Id	Cust_Name
601	Ram
603	Raj
NULL	Rani

Full join

```
SELECT Orders.Or_Id ,Customers.Cust_Name  
  
FROM Orders  
  
FULL JOIN Customers  
  
ON Orders.Cust_Id=Customers.Id;
```

Out put:

Or_Id	Cust_Name
601	Ram
602	NULL
603	Raj
NULL	Rani

Groupby and Having

In Hive, you can use the GROUP BY clause to group rows in a table based on one or more columns, and you can use the HAVING clause to filter the groups based on a condition. The GROUP BY clause is typically followed by an aggregation function, while the HAVING clause is used to filter groups based on the result of those aggregation functions. Here's an example to illustrate how to use GROUP BY and HAVING in Hive:

Assuming you have a table called **sales** with the following structure:

```
CREATE TABLE sales (  product_id INT, sale_date DATE,  
  
amount DECIMAL(10, 2));
```

And your table contains data like this:

product_id	sale_date	amount
1	2023-01-01	100.00
2	2023-01-01	150.00
1	2023-01-02	120.00
2	2023-01-02	180.00
1	2023-01-03	90.00
2	2023-01-03	160.00

Now, let's say you want to find products that have a total sales amount greater than 400. You can use GROUP BY to group the data by product_id and then use HAVING to filter the groups based on the sum of amount:

```
SELECT product_id,SUM(amount) AS total_sales
FROM sales
GROUP BY product_id
HAVING SUM(amount) > 400;
```

The result of this query will give you the products with a total sales amount greater than 400:

product_id	total_sales
2	490.00

in this example, the GROUP BY clause groups the data by product_id, and the SUM(amount) function calculates the total sales amount for each product group. The HAVING clause then filters out the groups where the total sales amount is not greater than 400.

RC File implementation

RCFile (Record Columnar File) is a columnar storage file format in Hadoop and Hive that is designed for efficient storage and querying of structured data. It is particularly useful when you have large datasets and need to perform analytical queries efficiently. RCFile stores data in a columnar format rather than a row-based format, which can lead to significant performance improvements for certain types of queries.

Here's an overview of how to implement RCFile in Hive:

Create a Table: First, you need to create a table in Hive that uses the RCFile format. You can specify the RCFile format by setting the STORED AS clause to 'RCFILE' when creating the table. Here's an example:

```
CREATE TABLE my_rcfile_table (  
    column1 INT,  
    column2 STRING,  
    column3 DOUBLE  
)  
  
ROW FORMAT DELIMITED  
  
FIELDS TERMINATED BY ','  
  
STORED AS RCFILE;
```

in this example, we've created a table named my_rcfile_table with three columns. The STORED AS RCFILE clause specifies that the data will be stored in the RCFile format.

Load Data: You can load data into your RCFile table just like any other Hive table. You can use the LOAD DATA command or insert data using INSERT INTO statements.

```
LOAD DATA LOCAL INPATH '/path/to/your/data' INTO TABLE my_rcfile_table;
```

Query Data: Once your data is loaded, you can query it using SQL in Hive. RCFile format is especially efficient for analytical queries because it stores data column-wise, allowing for better compression and faster column-based operations.

```
SELECT column1, AVG(column3)
```

```
FROM my_rcfile_table

WHERE column2 = 'some_condition'

GROUP BY column1;
```

Optimizing Performance: RCFile provides good performance for read-heavy workloads, but it may not be suitable for write-heavy workloads or frequent updates due to its columnar nature. To optimize performance further, you can consider partitioning your RCFile tables, using appropriate compression techniques, and tuning your Hive queries and configurations.

Data Maintenance: Regularly maintain your RCFile tables by optimizing them and removing unnecessary data to ensure efficient query performance.

RCFile is a valuable option for Hive when dealing with large datasets and analytical queries. However, it's essential to understand your specific use case and workload to determine whether RCFile is the right choice or if other storage formats like ORC (Optimized Row Columnar) or Parquet might be more suitable.

In Hive, User-Defined Functions (UDFs) allow you to extend the functionality of Hive by enabling you to write custom functions that can be used in Hive queries. UDFs are particularly useful when you need to perform operations that are not supported by Hive's built-in functions. You can create UDFs in Java, Python, or other programming languages supported by Hive.

Here are the steps to create and use a User-Defined Function in Hive:

Write the UDF Code: You need to write the code for your custom function in a programming language supported by Hive, such as Java or Python. The code should follow the UDF interface or class structure specific to your chosen language.

For example, let's say you want to create a simple UDF in Java that converts a string to uppercase:

```
import org.apache.hadoop.hive.ql.exec.UDF;

import org.apache.hadoop.io.Text;

public class UppercaseUDF extends UDF {
```

```

        public Text evaluate(Text input) {
            if (input == null) return null;

            return new Text(input.toString().toUpperCase());
        }
    }
}

```

Compile the UDF: You need to compile the UDF code into a JAR file or a shared library depending on the language you are using (e.g., a Java JAR file). Ensure that the compiled code is available on all nodes of your Hive cluster.

Register the UDF in Hive: Before you can use the UDF in Hive queries, you must register it using the CREATE FUNCTION statement:

```

CREATE FUNCTION uppercase AS 'com.example.UppercaseUDF' USING JAR
'path/to/your/udf.jar';

```

Here, uppercase is the name of the UDF, com.example.UppercaseUDF is the fully qualified class name, and 'path/to/your/udf.jar' is the path to the compiled JAR file containing the UDF code.

Use the UDF in Hive Queries: You can now use your custom UDF in Hive queries like any built-in function. For example:

```

SELECT uppercase('hello') AS result;

```

This query will return 'HELLO' as the result.

Cleanup and Maintenance: Be sure to manage your UDFs, including versioning and updating as needed. You can drop a UDF using the DROP FUNCTION statement when it's no longer required.

```

DROP FUNCTION uppercase;

```

User-Defined Functions in Hive provide great flexibility for custom data processing and transformation tasks. You can create UDFs for a wide range of use cases, including data cleansing, advanced calculations, and integration with external libraries or services. Depending on your specific needs, you can also explore other types of UDFs like User-Defined Aggregate

Functions (UDAFs) and User-Defined Table-Generating Functions (UDTFs) to further extend Hive's capabilities.

Serialization and Deserialization

In Hive, serialization and deserialization (often abbreviated as *serde*) are techniques used to convert data between its internal representation and external formats, such as when reading data from or writing data to files or external systems. Hive supports various serialization and deserialization formats, and you can choose the appropriate one based on your data requirements.

Here are the key concepts related to serialization and deserialization in Hive:

SerDe (Serializer/Deserializer): A SerDe is a library or module responsible for the conversion of data between Hive's internal data structures and external data formats. Hive supports various SerDes for different data formats, including CSV, JSON, Avro, Parquet, and more. Some common SerDes in Hive are *LazySimpleSerDe* for delimited text files and *AvroSerDe* for Avro data.

ROW FORMAT: When defining a table in Hive, you can specify the SerDe and related properties using the ROW FORMAT clause. This clause allows you to specify how data should be serialized when inserted into the table and deserialized when queried from the table. Here's an example:

```
CREATE TABLE my_table (  
    column1 INT,  
    column2 STRING  
)  
  
ROW                FORMAT                SERDE  
'org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe'  
  
WITH SERDEPROPERTIES (  
    'field.delim' = ','  
)  
  
STORED AS TEXTFILE;
```

In this example, we've used the LazySimpleSerDe SerDe to read and write data in a delimited text format (CSV). The `field.delim` property specifies that the delimiter between fields is a comma.

STORED AS: When creating a table, you can specify the storage format for the data using the `STORED AS` clause. The choice of file format (e.g., TEXTFILE, ORC, Parquet) affects how data is stored on disk. Different storage formats may require different SerDes for serialization and deserialization.

`STORED AS TEXTFILE;` -- Specifies that the data is stored as plain text files

`STORED AS ORC;` -- Specifies that the data is stored in the ORC columnar format

`STORED AS PARQUET;` -- Specifies that the data is stored in the Parquet columnar format.

Using SerDe in Queries: Once you've defined a table with a SerDe, you can use it in queries like any other table. Hive will automatically use the specified SerDe for data conversion.

Custom SerDes: Hive also allows you to create custom SerDes if you have specific data formats that are not covered by built-in SerDes. You can implement your own SerDe by implementing the SerDe interface and specifying it when creating a table.

Serialization and deserialization in Hive are essential for working with different data formats and sources. By selecting the appropriate SerDe and defining the correct `ROW FORMAT` and `STORED AS` clauses, you can seamlessly read and write data in various formats while interacting with Hive tables.

UNIT V

INTRODUCTION TO DATA ANALYTICS

Syllabus :

Machine Learning: Introduction, Supervised Learning, Unsupervised Learning, Machine Learning Algorithms: Regression Model, Clustering, Collaborative Filtering, Associate Rule Making, Decision Tree.

What is Machine Learning?

Machine learning is a subfield of artificial intelligence (AI) that focuses on the development of algorithms and statistical models that enable computers to learn and make predictions or decisions without being explicitly programmed. In other words, machine learning enables computers to improve their performance on a task by learning from data.

Here is an introduction to the key concepts and components of machine learning:

Data: Data is at the core of machine learning. Machine learning algorithms require a significant amount of data to learn patterns, make predictions, or take actions. Data can come in various forms, including structured data (e.g., databases), unstructured data (e.g., text, images, audio), and semi-structured data (e.g., JSON, XML).

Features: Features are the characteristics or attributes of the data that are used by machine learning algorithms for prediction or classification. For example, in a spam email detection system, features might include the words in an email and their frequencies.

Labels: Labels are the known outcomes or target values associated with a dataset. In supervised learning, a machine learning model learns from labeled data to make predictions or classifications. In unsupervised learning, there are no labels, and the model identifies patterns or clusters in the data.

Algorithms: Machine learning algorithms are mathematical models that learn from data and make predictions or decisions. There are various types of machine learning algorithms, including:

Supervised Learning: Algorithms learn from labeled data to predict or classify new data points. Examples include linear regression, decision trees, and support vector machines.

Unsupervised Learning: Algorithms find patterns or structures in unlabeled data. Examples include clustering algorithms like K-means and dimensionality reduction techniques like Principal Component Analysis (PCA).

Reinforcement Learning: Algorithms learn through trial and error to maximize a reward. It's often used in scenarios like game playing and autonomous robotics.

Deep Learning: Deep learning is a subset of machine learning that uses artificial neural networks with many layers (deep neural networks) to model complex patterns in data. It's particularly effective for tasks involving images, natural language, and speech recognition.

Training: In supervised learning, a machine learning model is trained using a dataset with known labels. During training, the model adjusts its parameters to minimize the difference between its predictions and the actual labels. This process is often referred to as "fitting" the model.

Testing and Validation: After training, the model's performance is evaluated on a separate dataset (test dataset) to assess its ability to generalize to new, unseen data. This step is crucial for ensuring that the model does not overfit the training data.

Deployment: Once a machine learning model has been trained and validated, it can be deployed in real-world applications to make predictions, automate decision-making, or enhance various processes.

Iterative Process: Machine learning is an iterative process that involves data collection, preprocessing, model selection and training, evaluation, and refinement. Continuous monitoring and retraining are often necessary to maintain the model's performance.

Machine learning has a wide range of applications across various domains, including healthcare, finance, marketing, image and speech recognition, autonomous vehicles, recommendation systems, and more. It is a rapidly evolving field with ongoing research and development, leading to advancements in AI capabilities

Types of Machine Learning

There are three main types of machine learning:

1. Supervised Learning
2. Unsupervised Learning
3. Reinforcement Learning

Supervised Learning

Supervised learning is a type of machine learning in which an algorithm learns from labeled data to make predictions or decisions. It is called "supervised" because the process of training the algorithm is analogous to having a teacher who provides guidance and feedback on the algorithm's predictions. In supervised learning, the algorithm learns to map input data to the correct output based on a set of training examples.

Key characteristics of supervised learning include:

Labeled Data: In supervised learning, the training dataset consists of pairs of input data (features) and their corresponding correct output (labels or target values). The labels represent the ground truth or the desired prediction for each input.

Learning from Examples: The algorithm learns patterns, relationships, or decision boundaries in the data by examining the labeled examples in the training dataset. It tries to capture the underlying mapping between inputs and outputs.

Prediction: Once the algorithm is trained, it can be used to make predictions or classifications on new, unseen data. The goal is to generalize from the training data to make accurate predictions for previously unseen instances.

Types of Problems: Supervised learning is commonly used for solving problems such as regression and classification:

Regression: In regression problems, the algorithm predicts a continuous numerical value. For example, predicting house prices based on features like square footage and the number of bedrooms is a regression task.

Classification: In classification problems, the algorithm assigns data points to predefined categories or classes. Examples include email spam detection (binary classification) and image recognition (multi-class classification).

Evaluation: The performance of a supervised learning model is typically evaluated using various metrics, depending on the type of problem. Common evaluation metrics for classification tasks include accuracy, precision, recall, F1-score, and area under the ROC curve (AUC). For regression tasks, metrics like mean squared error (MSE) and R-squared are used.

Common algorithms used in supervised learning include:

Linear Regression: Used for regression tasks when there is a linear relationship between the input features and the target variable.

Logistic Regression: Used for binary and multi-class classification tasks.

Decision Trees: Versatile for both classification and regression tasks. They create a tree-like structure to make decisions based on feature values.

Random Forest: An ensemble method that combines multiple decision trees to improve predictive accuracy.

Support Vector Machines (SVM): Effective for classification tasks, particularly when there is a clear margin of separation between classes.

k-Nearest Neighbors (k-NN): Assigns labels based on the majority class among the k-nearest data points.

Neural Networks: Deep learning models, such as artificial neural networks, are used for complex tasks, including image and speech recognition.

Supervised learning is widely applied in various domains, including natural language processing, computer vision, healthcare, finance, and many others. It requires labeled data for training, and the quality and quantity of the training data can significantly impact the performance of the learned model.

Unsupervised learning

Unsupervised learning is a type of machine learning where the algorithm learns from data that is not labeled with explicit target values or categories. Unlike supervised learning, where the algorithm is guided by labeled examples, unsupervised learning aims to discover patterns, structures, or relationships within the data without any predefined supervision. Unsupervised learning is often used for exploratory data analysis and finding hidden insights within datasets.

Key characteristics of unsupervised learning include:

Unlabeled Data: In unsupervised learning, the training dataset consists of raw data with no corresponding labels or target values. The algorithm's goal is to find intrinsic patterns or groupings within the data.

Clustering: Clustering is one of the primary tasks in unsupervised learning. Clustering algorithms group similar data points together based on their feature similarities, without prior

knowledge of the classes or categories. K-means clustering and hierarchical clustering are common clustering techniques.

Dimensionality Reduction: Another common task in unsupervised learning is dimensionality reduction. Dimensionality reduction techniques aim to reduce the number of features (attributes) in the data while preserving essential information. Principal Component Analysis (PCA) is a widely used dimensionality reduction technique.

Anomaly Detection: Unsupervised learning can also be used for anomaly detection, where the algorithm identifies data points that deviate significantly from the norm or the majority of the data. Anomalies may represent unusual or potentially problematic instances, such as fraud in financial transactions or defects in manufacturing.

Density Estimation: Unsupervised learning can be used to estimate the probability density function of the data, allowing for the modeling of data distribution.

Representation Learning: Some unsupervised learning algorithms focus on learning useful representations of data. These learned representations can be used as input features for downstream supervised learning tasks.

Common algorithms used in unsupervised learning include:

K-means Clustering: A partitioning algorithm that divides data into k clusters based on the similarity of data points.

Hierarchical Clustering: Builds a tree-like structure of clusters, which can be visualized as a dendrogram, showing the hierarchical relationships between data points.

Principal Component Analysis (PCA): Reduces the dimensionality of data while preserving as much variance as possible.

Autoencoders: Neural network architectures used for representation learning by encoding and decoding data.

Gaussian Mixture Models (GMMs): Models data as a mixture of several Gaussian distributions, useful for density estimation.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise): Identifies clusters based on data density, suitable for non-uniformly distributed data.

Applications of unsupervised learning include:

Customer segmentation in marketing

- Natural language processing (e.g., topic modeling)
- Image and document clustering
- Anomaly detection in cybersecurity
- Reducing dimensionality in high-dimensional datasets
- Discovering hidden patterns in data for scientific research

Unsupervised learning is valuable for data exploration and can reveal insights into complex datasets where the underlying structure or relationships are not well understood

Reinforcement Learning (RL)

Reinforcement Learning (RL) is a type of machine learning where an agent learns to make a sequence of decisions by interacting with an environment in order to maximize a cumulative reward. It is inspired by the concept of trial and error learning, similar to how humans and animals learn from their experiences.

Key components and concepts of reinforcement learning include:

Agent: The agent is the learner or decision-maker in the RL framework. It interacts with the environment and takes actions to achieve its goals. The agent can be a software program, a robot, or any entity that learns and makes decisions.

Environment: The environment represents the external system with which the agent interacts. It includes all the elements, rules, and dynamics that the agent needs to consider when taking actions. The environment provides feedback to the agent in the form of rewards or penalties based on its actions.

State: A state represents the current situation or configuration of the environment. It contains all the relevant information needed to make decisions. States can be discrete or continuous, depending on the problem.

Action: Actions are the choices made by the agent to influence the environment. Actions can be discrete (e.g., choosing from a set of predefined actions) or continuous (e.g., controlling motor speeds). The set of all possible actions is known as the action space.

Policy: A policy is a strategy or a mapping from states to actions. It defines how the agent chooses its actions in different states. The goal of RL is often to find an optimal policy that maximizes the expected cumulative reward.

Reward: A reward is a numerical signal provided by the environment to the agent after each action. It indicates the immediate desirability of the action taken. The agent's objective is to maximize the cumulative reward over time.

Value Function: The value function is a function that estimates the expected cumulative reward the agent can achieve starting from a particular state or state-action pair. It helps the agent evaluate the quality of states and actions.

Exploration vs. Exploitation: One of the key challenges in RL is the trade-off between exploration (trying new actions to learn more about the environment) and exploitation (choosing actions that are known to yield high rewards). Finding the right balance is crucial for effective learning.

Popular algorithms and techniques used in reinforcement learning include:

Q-Learning: Q-learning is a model-free RL algorithm that learns the quality (Q-value) of state-action pairs. It can be used for discrete action spaces.

Deep Q-Networks (DQN): DQN is an extension of Q-learning that uses deep neural networks to approximate the Q-value function. It is suitable for high-dimensional state spaces.

Policy Gradients: Policy gradient methods directly optimize the policy to maximize expected rewards. They are often used in continuous action spaces.

Actor-Critic: Actor-critic methods combine elements of both policy gradients and value function estimation. They have separate actors (policies) and critics (value functions).

Monte Carlo Methods: These methods estimate value functions or policies by averaging returns from sampled trajectories.

Proximal Policy Optimization (PPO): PPO is a policy optimization algorithm that helps stabilize training and prevent large policy updates.

Recurrent Neural Networks (RNNs): RNNs are used to model sequential decision-making problems, where the state transitions depend on previous actions and observations.

Applications of reinforcement learning include:

- Game playing (e.g., AlphaGo, video games)
- Robotics and autonomous systems
- Recommendation systems
- Healthcare (e.g., personalized treatment plans)
- Finance (e.g., portfolio optimization)
- Natural language processing
- Autonomous vehicles

Reinforcement learning is a powerful paradigm for solving sequential decision-making problems where the optimal strategy may not be known in advance. It has led to significant advancements in various domains and continues to be an active area of research and development.

Regression Analysis in Machine learning

A regression machine learning model is a type of model used for predictive modeling when the target variable (the variable you want to predict) is continuous or numeric. In regression analysis, the goal is to learn a mathematical relationship between input features and the continuous target variable. The learned model can then be used to make predictions on new data points.

Terminologies Related to the Regression Analysis:

Dependent Variable: The main factor in Regression analysis which we want to predict or understand is called the dependent variable. It is also called target variable.

Independent Variable: The factors which affect the dependent variables or which are used to predict the values of the dependent variables are called independent variable, also called as a predictor.

Outliers: Outlier is an observation which contains either very low value or very high value in comparison to other observed values. An outlier may hamper the result, so it should be avoided.

Multicollinearity: If the independent variables are highly correlated with each other than other variables, then such condition is called Multicollinearity. It should not be present in the dataset, because it creates problem while ranking the most affecting variable.

Underfitting and Overfitting: If our algorithm works well with the training dataset but not well with test dataset, then such problem is called Overfitting. And if our algorithm does not perform well even with training dataset, then such problem is called underfitting.

Why do we use Regression Analysis?

As mentioned above, Regression analysis helps in the prediction of a continuous variable. There are various scenarios in the real world where we need some future predictions such as weather condition, sales prediction, marketing trends, etc., for such case we need some technology which can make predictions more accurately. So for such case we need Regression analysis which is a statistical method and used in machine learning and data science.

Types of Regression

There are various types of regressions which are used in data science and machine learning. Each type has its own importance on different scenarios, but at the core, all the regression methods analyze the effect of the independent variable on dependent variables. Here we are discussing some important types of regression which are given below:

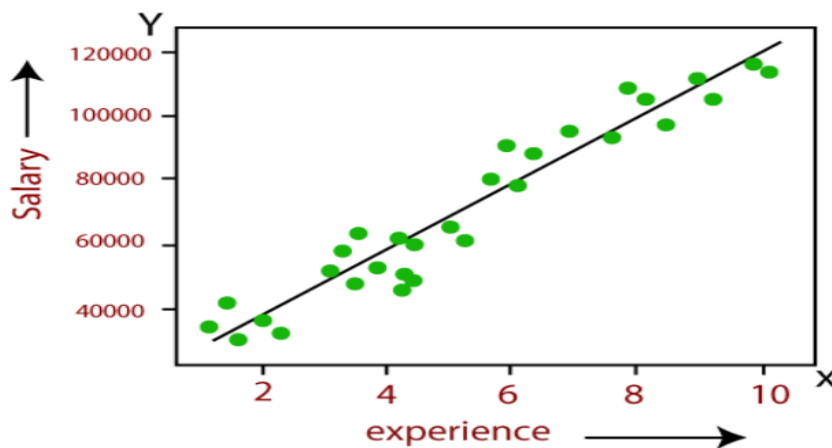
- . Linear Regression
- Logistic Regression
- Polynomial Regression
- Support Vector Regression
- Decision Tree Regression
- Random Forest Regression
- Ridge Regression
- Lasso Regression

Linear Regression:

Simple Linear Regression: In simple linear regression, there is one independent variable and one dependent variable, and the relationship between them is assumed to be linear. It aims to fit a straight line (linear equation) to the data to make predictions.

Multiple Linear Regression: Multiple linear regression extends simple linear regression to multiple independent variables. It models the relationship between multiple predictors and a single dependent variable through a linear equation.

o The relationship between variables in the linear regression model can be explained using the below image. Here we are predicting the salary of an employee on the basis of the year of experience.



Below is the mathematical equation for Linear regression:

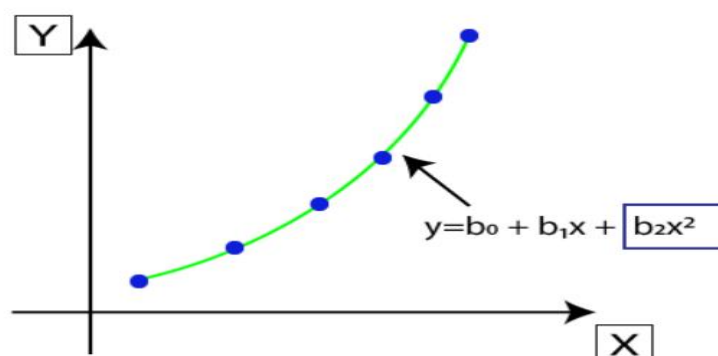
$$Y = aX + b$$

Here, Y = dependent variables (target variables), X = Independent variables (predictor variables), a and b are the linear coefficients.

Polynomial Regression:

Polynomial regression is an extension of linear regression that allows for modeling nonlinear relationships between the independent and dependent variables. It fits a polynomial curve (e.g., quadratic or cubic) to the data.

The equation for polynomial regression also derived from linear regression equation that means Linear regression equation $Y = b_0 + b_1x$, is transformed into Polynomial regression equation $Y = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_nx^n$.



Logistic Regression:

Logistic regression is used for binary classification problems, where the dependent variable is binary (e.g., 0 or 1, Yes or No). It models the probability of a data point belonging to one of the two classes using the logistic function.

Logistic regression uses sigmoid function or logistic function which is a complex cost function. This sigmoid function is used to model the data in logistic regression.

The function can be represented as:

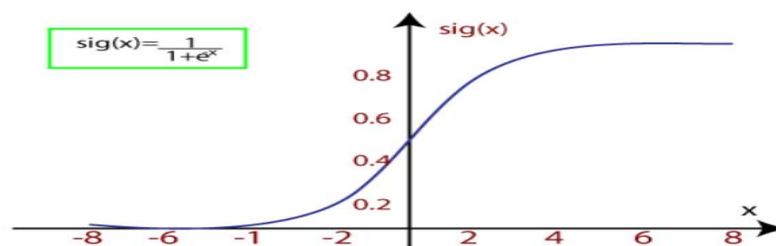
$$f(x) = \frac{1}{1 + e^{-x}}$$

$f(x)$ = Output between the 0 and 1 value.

x = input to the function

e = base of natural logarithm

When we provide the input values (data) to the function, it gives the S-curve as follows



Ridge Regression and Lasso Regression:

Ridge and Lasso regression are extensions of linear regression that introduce regularization terms to the linear equation. They are used to handle multicollinearity (correlation between predictors) and prevent overfitting.

Clustering in Machine Learning

Clustering or cluster analysis is a machine learning technique, which groups the unlabelled dataset. It can be defined as "A way of grouping the data points into different clusters, consisting of similar data points. The objects with the possible similarities remain in a group that has less or no similarities with another group."

It does it by finding some similar patterns in the unlabelled dataset such as shape, size, color, behavior, etc., and divides them as per the presence and absence of those similar patterns.

It is an unsupervised learning method, hence no supervision is provided to the algorithm, and it deals with the unlabeled dataset.

After applying this clustering technique, each cluster or group is provided with a cluster-ID. ML system can use this id to simplify the processing of large and complex datasets

The clustering technique is commonly used for statistical data analysis.

Note: Clustering is somewhere similar to the classification algorithm, but the difference is the type of dataset that we are using. In classification, we work with the labeled data set, whereas in clustering, we work with the unlabelled dataset.

Example: Let's understand the clustering technique with the real-world example of Mall: When we visit any shopping mall, we can observe that the things with similar usage are grouped together. Such as the t-shirts are grouped in one section, and trousers are at other sections, similarly, at vegetable sections, apples, bananas, Mangoes, etc., are grouped in separate sections, so that we can easily find out the things. The clustering technique also works in the same way. Other examples of clustering are grouping documents according to the topic.

The clustering technique can be widely used in various tasks. Some most common uses of this technique are:

- o Market Segmentation
- o Statistical data analysis
- o Social network analysis o Image segmentation o Anomaly detection, etc.

Apart from these general usages, it is used by the Amazon in its recommendation system to provide the recommendations as per the past search of products. Netflix also uses this technique to recommend the movies and web-series to its users as per the watch history.

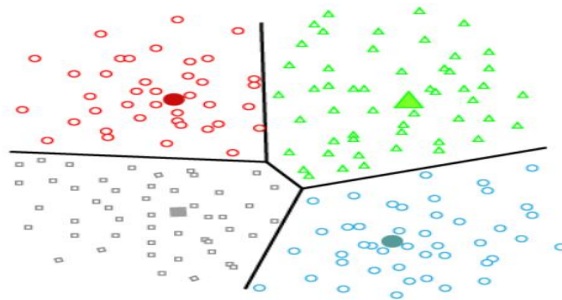
Types of Clustering Methods

The clustering methods are broadly divided into Hard clustering (datapoint belongs to only one group) and Soft Clustering (data points can belong to another group also). But there are also other various approaches of Clustering exist. Below are the main clustering methods used in Machine learning:

- 1.Partitioning Clustering
2. Density-Based Clustering
3. Distribution Model-Based Clustering
4. Hierarchical Clustering
5. Fuzzy Clustering

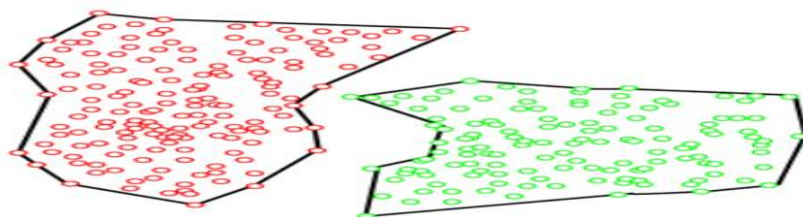
Partitioning Clustering

It is a type of clustering that divides the data into non-hierarchical groups. It is also known as the centroid-based method. The most common example of partitioning clustering is the K-Means Clustering algorithm. In this type, the dataset is divided into a set of k groups, where K is used to define the number of pre-defined groups. The cluster center is created in such a way that the distance between the data points of one cluster is minimum as compared to another cluster centroid.



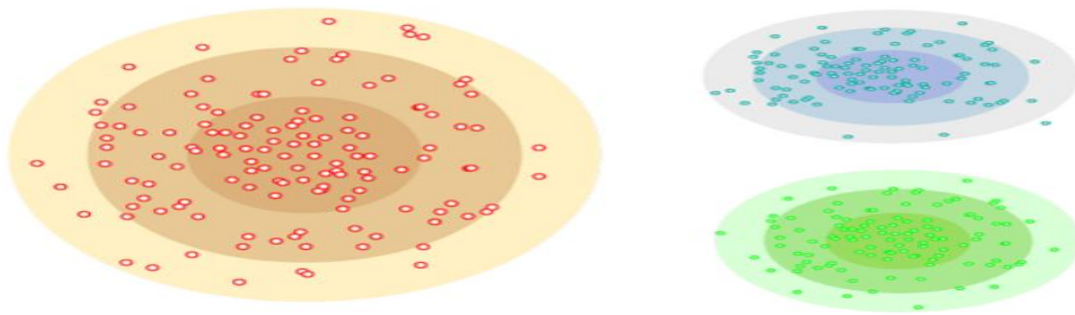
Density-Based Clustering

The density-based clustering method connects the highly-dense areas into clusters, and the arbitrarily shaped distributions are formed as long as the dense region can be connected. This algorithm does it by identifying different clusters in the dataset and connects the areas of high densities into clusters. The dense areas in data space are divided from each other by sparser areas. These algorithms can face difficulty in clustering the data points if the dataset has varying densities and high dimensions.



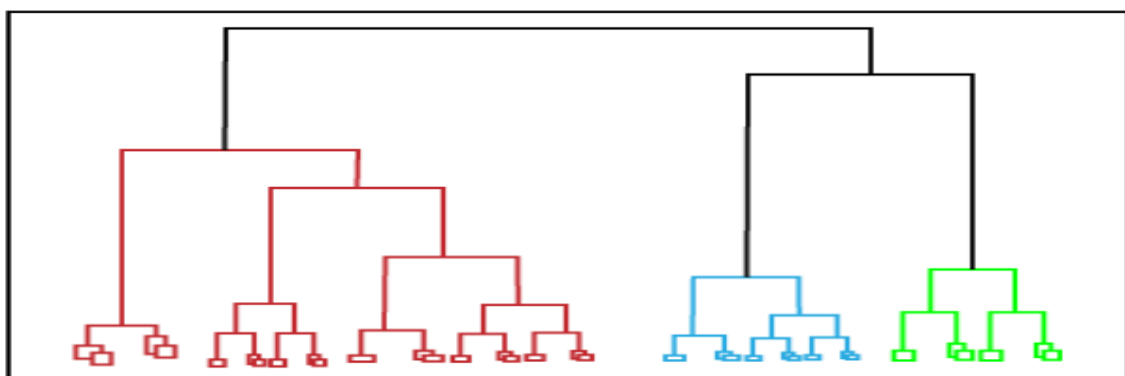
Distribution Model-Based Clustering

In the distribution model-based clustering method, the data is divided based on the probability of how a dataset belongs to a particular distribution. The grouping is done by assuming some distributions commonly Gaussian Distribution. The example of this type is the Expectation-Maximization Clustering algorithm that uses Gaussian Mixture Models (GMM).



Hierarchical Clustering

Hierarchical clustering can be used as an alternative for the partitioned clustering as there is no requirement of pre-specifying the number of clusters to be created. In this technique, the dataset is divided into clusters to create a tree-like structure, which is also called a dendrogram. The observations or any number of clusters can be selected by cutting the tree at the correct level. The most common example of this method is the Agglomerative Hierarchical algorithm.



Fuzzy Clustering

Fuzzy clustering is a type of soft method in which a data object may belong to more than one group or cluster. Each dataset has a set of membership coefficients, which depend on the degree

of membership to be in a cluster. Fuzzy C-means algorithm is the example of this type of clustering; it is sometimes also known as the Fuzzy k-means algorithm.

Clustering Algorithms

The Clustering algorithms can be divided based on their models that are explained above. There are different types of clustering algorithms published, but only a few are commonly used. The clustering algorithm is based on the kind of data that we are using. Such as, some algorithms need to guess the number of clusters in the given dataset, whereas some are required to find the minimum distance between the observation of the dataset.

Here we are discussing mainly popular Clustering algorithms that are widely used in machine learning.

1. K-Means algorithm: The k-means algorithm is one of the most popular clustering algorithms. It classifies the dataset by dividing the samples into different clusters of equal variances. The number of clusters must be specified in this algorithm. It is fast with fewer computations required, with the linear complexity of $O(n)$.

2. Mean-shift algorithm: Mean-shift algorithm tries to find the dense areas in the smooth density of data points. It is an example of a centroid-based model, that works on updating the candidates for centroid to be the center of the points within a given region.

3. DBSCAN Algorithm: It stands for Density-Based Spatial Clustering of Applications with Noise. It is an example of a density-based model similar to the mean-shift, but with some remarkable advantages. In this algorithm, the areas of high density are separated by the areas of low density. Because of this, the clusters can be found in any arbitrary shape.

4. Expectation-Maximization Clustering using GMM: This algorithm can be used as an alternative for the k-means algorithm or for those cases where K-means can be failed. In GMM, it is assumed that the data points are Gaussian distributed.

5. Agglomerative Hierarchical algorithm: The Agglomerative hierarchical algorithm performs the bottom-up hierarchical clustering. In this, each data point is treated as a single cluster at the outset and then successively merged. The cluster hierarchy can be represented as a tree-structure.

6. Affinity Propagation: It is different from other clustering algorithms as it does not require to specify the number of clusters. In this, each data point sends a message between the pair of

data points until convergence. It has $O(N^2 T)$ time complexity, which is the main drawback of this algorithm.

Collaborative filtering

Collaborative filtering is a popular technique used in recommendation systems to provide personalized recommendations to users. It relies on user behavior and preferences, often collected through user interactions, to make predictions about what products or items a user might be interested in. Collaborative filtering is based on the idea that users who have interacted or rated items similarly in the past are likely to have similar preferences in the future.

There are two main approaches to collaborative filtering:

User-Based Collaborative Filtering:

- In user-based collaborative filtering, recommendations are made by identifying users who are similar to the target user based on their past interactions or ratings.
- The process involves finding a set of users (neighbors) whose past behavior closely matches that of the target user.
- Recommendations are then generated based on the items liked or rated highly by the neighbors but not yet seen by the target user.
- Common similarity metrics for user-based collaborative filtering include cosine similarity, Pearson correlation, and Jaccard similarity.

Item-Based Collaborative Filtering:

- In item-based collaborative filtering, recommendations are made by identifying items that are similar to the items the target user has already interacted with or rated.
- The process involves calculating item-item similarities based on user behavior (e.g., item co-occurrence, item ratings).
- Recommendations are generated by suggesting items similar to those the user has already shown an interest in.
- Item-based collaborative filtering is often more scalable than user-based approaches because item-item similarity matrices are typically smaller than user-user similarity matrices.

Collaborative filtering has several advantages:

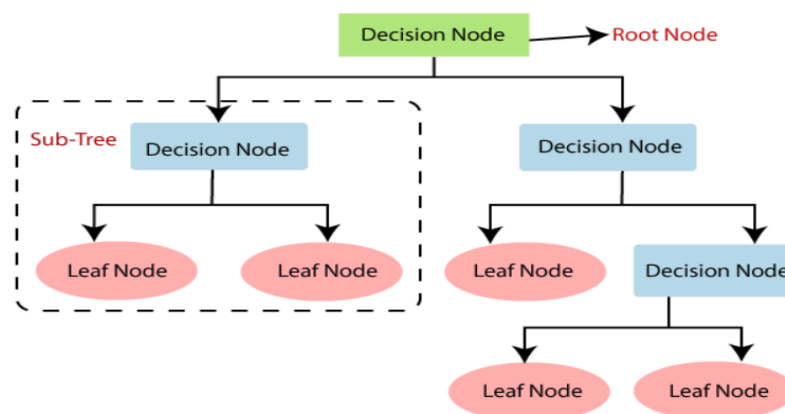
- It can provide accurate recommendations even when little or no information about items is available (cold-start problem).
- It can capture complex user preferences and patterns.
- It can adapt to changes in user behavior over time.
- However, collaborative filtering also has limitations:
- It requires a sufficient amount of user interaction data to make accurate recommendations.
- It may suffer from the "sparsity" problem when the number of users or items is large, and users have interacted with only a small subset of items.
- It can be sensitive to data quality issues, such as noisy or biased ratings.
- To address these limitations, hybrid recommendation systems are often used, combining collaborative filtering with other techniques like content-based filtering or matrix factorization.
- Collaborative filtering is widely used in various recommendation systems, including movie recommendations, e-commerce product recommendations, music playlists, and more. It has proven to be effective in providing personalized experiences to users and enhancing user engagement.

Decision Tree Classification Algorithm

- Decision Tree is a Supervised learning technique that can be used for both classification and Regression problems, but mostly it is preferred for solving Classification problems. It is a tree structured classifier, where internal nodes represent the features of a dataset, branches represent the decision rules and each leaf node represents the outcome.
- In a Decision tree, there are two nodes, which are the Decision Node and Leaf Node. Decision nodes are used to make any decision and have multiple branches, whereas Leaf nodes are the output of those decisions and do not contain any further branches.
- The decisions or the test are performed on the basis of features of the given dataset.
 - o It is a graphical representation for getting all the possible solutions to a problem/decision based on given conditions.
- It is called a decision tree because, similar to a tree, it starts with the root node, which expands on further branches and constructs a tree-like structure.

- In order to build a tree, we use the CART algorithm, which stands for Classification and Regression Tree algorithm.
- A decision tree simply asks a question, and based on the answer (Yes/No), it further split the tree into subtrees.
- Below diagram explains the general structure of a decision tree:

Note: A decision tree can contain categorical data (YES/NO) as well as numeric data.



Why use Decision Trees?

There are various algorithms in Machine learning, so choosing the best algorithm for the given dataset and problem is the main point to remember while creating a machine learning model. Below are the two reasons for using the Decision tree:

- Decision Trees usually mimic human thinking ability while making a decision, so it is easy to understand.
- The logic behind the decision tree can be easily understood because it shows a tree-like structure

Decision Tree Terminologies

- **Root Node:** Root node is from where the decision tree starts. It represents the entire dataset, which further gets divided into two or more homogeneous sets.

- **Leaf Node:** Leaf nodes are the final output node, and the tree cannot be segregated further after getting a leaf node.

- **Splitting:** Splitting is the process of dividing the decision node/root node into sub-nodes according to the given conditions.

- **Branch/Sub Tree:** A tree formed by splitting the tree.
- **Pruning:** Pruning is the process of removing the unwanted branches from the tree.

- **Parent/Child node:** The root node of the tree is called the parent node, and other nodes are called the child nodes

How does the Decision Tree algorithm Work?

In a decision tree, for predicting the class of the given dataset, the algorithm starts from the root node of the tree. This algorithm compares the values of root attribute with the record (real dataset) attribute and, based on the comparison, follows the branch and jumps to the next node. For the next node, the algorithm again compares the attribute value with the other sub-nodes and move further. It continues the process until it reaches the leaf node of the tree. The complete process can be better understood using the below algorithm

Step-1: Begin the tree with the root node, says S, which contains the complete dataset.

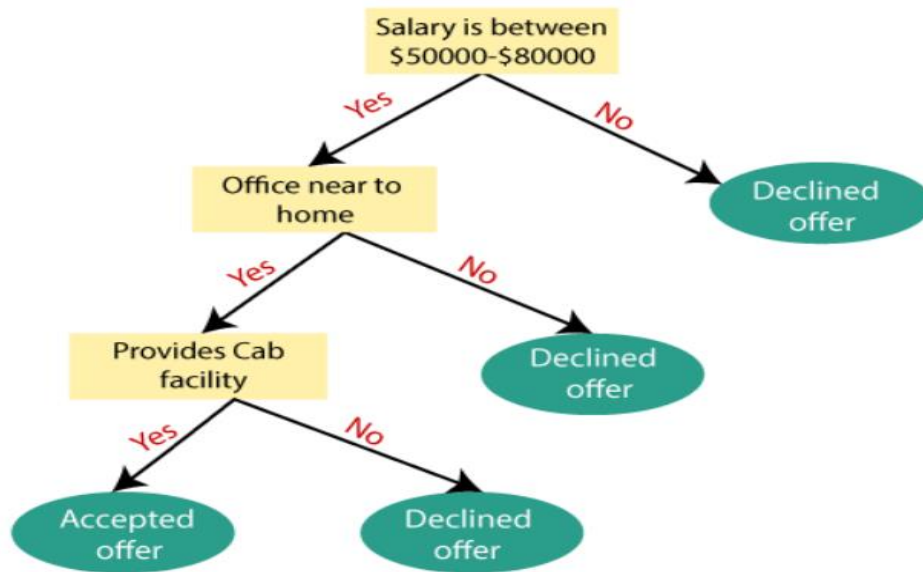
Step-2: Find the best attribute in the dataset using Attribute Selection Measure (ASM).

Step-3: Divide the S into subsets that contains possible values for the best attributes.

Step-4: Generate the decision tree node, which contains the best attribute.

Step-5: Recursively make new decision trees using the subsets of the dataset created in step - 3. Continue this process until a stage is reached where you cannot further classify the nodes and called the final node as a leaf node.

Example: Suppose there is a candidate who has a job offer and wants to decide whether he should accept the offer or Not. So, to solve this problem, the decision tree starts with the rootnode (Salary attribute by ASM). The root node splits further into the next decision node (distance from the office) and one leaf node based on the corresponding labels. The next decision node further gets split into one decision node (Cab facility) and one leaf node. Finally, the decision node splits into two leaf nodes (Accepted offers and Declined offer). Consider the below diagram:



Attribute Selection Measures (ASM)

While implementing a Decision tree, the main issue arises that how to select the best attribute for the root node and for sub-nodes. So, to solve such problems there is a technique which is called as Attribute selection measure or ASM. By this measurement, we can easily select the best attribute for the nodes of the tree. There are two popular techniques for ASM, which are

- Information Gain
- Gini Index

1. Information Gain:

- o Information gain is the measurement of changes in entropy after the segmentation of a dataset based on an attribute.
- o It calculates how much information a feature provides us about a class.
- o According to the value of information gain, we split the node and build the decision tree.
- o A decision tree algorithm always tries to maximize the value of information gain, and a node/attribute having the highest information gain is split first. It can be calculated using the below formula:

$$\text{Information Gain} = \text{Entropy}(S) - [(\text{Weighted Avg}) * \text{Entropy}(\text{each feature})]$$

Entropy: Entropy is a metric to measure the impurity in a given attribute. It specifies randomness in data. Entropy can be calculated as:

$$\text{Entropy}(s) = -P(\text{yes}) \log_2 P(\text{yes}) - P(\text{no}) \log_2 P(\text{no})$$

Where,

S= Total number of samples

P(yes)= probability of yes

P(no)= probability of no

2. Gini Index:

- o Gini index is a measure of impurity or purity used while creating a decision tree in the CART(Classification and Regression Tree) algorithm.
- o An attribute with the low Gini index should be preferred as compared to the high Gini index.
- o It only creates binary splits, and the CART algorithm uses the Gini index to create binary splits.
- o Gini index can be calculated using the below formula:

$$\text{Gini Index} = 1 - \sum_j P_j^2$$

Pruning: Getting an Optimal Decision tree Pruning is a process of deleting the unnecessary nodes from a tree in order to get the optimal decision tree. A too-large tree increases the risk of overfitting, and a small tree may not capture all the important features of the dataset.

Therefore, a technique that decreases the size of the learning tree without reducing accuracy is known as Pruning. There are mainly two types of tree pruning technology used:

- o Cost Complexity Pruning
- o Reduced Error Pruning